

Teoría de Grafos y Algoritmos Fundamentales

Un enfoque práctico para desarrolladores y matemáticos

Alejandro Lazaro Alfonso Yero
Colaboración Técnica: Gemini 3 Pro (AI)

2025

Índice

1	Fundamentos de Grafos y Representaciones Eficientes	1
1.1	Introducción a los Grafos: ¿Por Qué Son Tan Ubicuos?	1
1.1.1	Contexto Histórico y Orígenes	1
1.1.2	El Mundo es un Grafo: Ejemplos Intuitivos y Avanzados	1
1.2	Definiciones Fundamentales y Terminología	2
1.2.1	Componentes Básicos	2
1.2.2	Tipos de Grafos	2
1.2.3	Terminología de Vértices y Aristas	5
1.2.4	Caminos y Conectividad	5
1.3	Representaciones Computacionales de Grafos	6
1.3.1	Consideraciones de Diseño	6
1.3.2	Lista de Adyacencia (Adjacency List)	6
1.3.3	Matriz de Adyacencia (Adjacency Matrix)	8
1.3.4	Lista de Aristas (Edge List)	10
1.4	Comparación y Elección de Representaciones	12
1.4.1	Tabla Comparativa Detallada	12
1.4.2	Casos de Uso	13
1.4.3	Librerías de Grafos en Python (Introducción)	13
2	Recorridos y Búsqueda en Grafos	15
2.1	2.1. Introducción a los Recorridos en Grafos	15
2.1.1	2.1.1. ¿Qué es un Recorrido?	15
2.1.2	2.1.2. Aplicaciones Fundamentales	15
2.2	2.2. Búsqueda en Amplitud (BFS - Breadth-First Search)	16
2.2.1	2.2.1. Concepto y Funcionamiento	16
2.2.2	2.2.2. Algoritmo Detallado	16
2.2.3	2.2.3. Implementación en Python	17
2.2.4	2.2.4. Análisis de Complejidad	18
2.2.5	2.2.5. Aplicaciones de BFS	18
2.3	2.3. Búsqueda en Profundidad (DFS - Depth-First Search)	18
2.3.1	2.3.1. Concepto y Funcionamiento	18
2.3.2	2.3.2. Algoritmo Detallado	18
2.3.3	2.3.3. Implementación en Python	19
2.3.4	2.3.4. Análisis de Complejidad	21
2.3.5	2.3.5. Aplicaciones de DFS	21
3	Caminos Más Cortos: Algoritmos Clásicos	23
3.1	Introducción al Problema del Camino Más Corto	23
3.1.1	Definición del Problema	23
3.1.2	Tipos de Grafos y Consideraciones	23

3.2	Algoritmo de Dijkstra	24
3.2.1	Concepto y Funcionamiento	24
3.2.2	Algoritmo Detallado	24
3.2.3	Implementación en Python	24
3.2.4	Análisis de Complejidad	26
3.2.5	Limitaciones	26
3.3	Algoritmo de Bellman-Ford	27
3.3.1	Concepto y Funcionamiento	27
3.3.2	Algoritmo Detallado	27
3.3.3	Implementación en Python	27
3.3.4	Análisis de Complejidad	29
3.3.5	Ventajas y Desventajas	29
3.4	Algoritmo de Floyd-Warshall	29
3.4.1	Concepto y Funcionamiento	29
3.4.2	Algoritmo Detallado	30
3.4.3	Implementación en Python	30
3.4.4	Análisis de Complejidad	32
3.4.5	Ventajas y Desventajas	32
3.5	Algoritmo A* (A-Star)	32
3.5.1	Concepto y Funcionamiento	32
3.5.2	Algoritmo Detallado	32
3.5.3	Implementación en Python	33
3.5.4	Análisis de Complejidad	35
3.5.5	Propiedades y Usos	35
4	Árboles de Expansión Mínima (MST)	37
4.1	Introducción a los Árboles de Expansión Mínima	37
4.1.1	¿Qué es un Árbol de Expansión?	37
4.1.2	El Problema del Árbol de Expansión Mínima (MST)	37
4.1.3	Propiedades Clave del MST	38
4.2	Algoritmo de Prim	38
4.2.1	Concepto y Funcionamiento	38
4.2.2	Algoritmo Detallado	39
4.2.3	Implementación en Python	39
4.2.4	Análisis de Complejidad	41
4.2.5	Limitaciones	41
4.3	Algoritmo de Kruskal	41
4.3.1	Concepto y Funcionamiento	41
4.3.2	La Estructura de Datos Union-Find (DSU)	42
4.3.3	Implementación de Kruskal en Python	43
4.3.4	Análisis de Complejidad	45
4.3.5	Comparativa Prim vs. Kruskal	45
5	Grafos Dirigidos Acíclicos (DAGs) y Ordenamiento Topológico	47
5.1	Introducción a los DAGs	47
5.1.1	Definición y Propiedades	47
5.2	Ordenamiento Topológico	47
5.2.1	Algoritmo de Kahn (Basado en Grados de Entrada)	48
5.2.2	Algoritmo Basado en DFS	49
5.3	Aplicaciones Prácticas	50
5.3.1	Resolución de Dependencias	50
5.3.2	Planificación de Tareas (Scheduling)	50

5.4	Caminos Más Largos y Ruta Crítica	50
5.4.1	Algoritmo usando Ordenamiento Topológico	50
6	Conectividad y Componentes Fuertemente Conexas (SCC)	53
6.1	Conectividad en Grafos No Dirigidos	53
6.1.1	Conceptos Clave	53
6.1.2	Algoritmo para Encontrar Puentes (Tarjan)	53
6.2	Conectividad en Grafos Dirigidos	55
6.2.1	Componentes Fuertemente Conexas (SCC)	55
6.2.2	Algoritmo de Kosaraju	55
6.3	Aplicaciones Prácticas	57
6.3.1	Análisis de Redes y Vulnerabilidad	57
6.3.2	Resolución de 2-Satisfacibilidad (2-SAT)	57
6.3.3	Optimización de Consultas en Grafos Web	57
7	Flujo Máximo y Matching	59
7.1	Redes de Flujo	59
7.1.1	Propiedades del Flujo	59
7.2	Algoritmo de Ford-Fulkerson	60
7.2.1	Conceptos Clave	60
7.2.2	Algoritmo Detallado	60
7.3	Algoritmo de Edmonds-Karp	60
7.3.1	Implementación en Python	61
7.4	Matching en Grafos Bipartitos	62
7.4.1	Aplicaciones	63
7.4.2	Reducción a Flujo Máximo	63
7.4.3	Teorema de Flujo Máximo - Corte Mínimo	63
7.4.4	Implementación de Matching Bipartito (vía Flujo)	63
8	Grafos Especiales y sus Algoritmos	65
8.1	Árboles	65
8.1.1	Propiedades Definitorias	65
8.1.2	Terminología de Árboles Enraizados	65
8.1.3	Recorridos en Árboles (Tree Traversals)	66
8.1.4	Centro y Diámetro de un Árbol	67
8.2	Grafos Bipartitos	67
8.2.1	Caracterización	67
8.2.2	Detección de Bipartición (2-Coloreado)	67
8.3	Grafos Planos	68
8.3.1	Fórmula de Euler	68
8.3.2	Teorema de Kuratowski	68
8.3.3	Teorema de los 4 Colores	68
8.3.4	Aplicaciones	69
9	Introducción a la Complejidad y Técnicas Avanzadas	71
9.1	Clases de Complejidad: P, NP y NP-Completo	71
9.1.1	Definiciones Básicas	71
9.1.2	Problemas NP-Completo Famosos en Grafos	71
9.1.3	¿Qué hacer ante un problema NP-Completo?	72
9.2	2-Satisfiability (2-SAT)	72
9.2.1	Formulación	72
9.2.2	Construcción del Grafo de Implicación	72

9.2.3 Solución con Componentes Fuertemente Conexas (SCC)	72
9.3 Heavy-Light Decomposition (HLD)	74
9.3.1 Concepto	74
9.3.2 Aplicaciones	74
9.4 Grafos Aleatorios y Redes de Pequeño Mundo	74
10 Aplicaciones Prácticas y Estudios de Caso	77
10.1 Análisis de Redes Sociales (Social Network Analysis - SNA)	77
10.1.1 Métricas de Centralidad	77
10.1.2 Detección de Comunidades	78
10.2 Mapas y Navegación	78
10.2.1 Modelado del Problema	78
10.2.2 Algoritmos en Producción	78
10.3 Compiladores y Sistemas Operativos	78
10.3.1 Resolución de Dependencias	79
10.3.2 Asignación de Registros (Register Allocation)	79
10.4 Biología Computacional	79
10.4.1 Ensamblaje de Genomas	79
10.4.2 Redes de Interacción de Proteínas (PPI)	79
10.5 Conclusión	80

Capítulo 1

Fundamentos de Grafos y Representaciones Eficientes

Este capítulo introduce los conceptos fundamentales de la teoría de grafos, explorando su importancia en diversos campos de la computación y presentando las formas más comunes y eficientes de representarlos en código, con un enfoque particular en Python.

1.1 Introducción a los Grafos: ¿Por Qué Son Tan Ubicuos?

La teoría de grafos no es una abstracción puramente matemática; es un marco conceptual que subyace a innumerables estructuras y problemas en el mundo real y la computación. Comprender los grafos es fundamental para cualquier desarrollador de software que busque modelar sistemas complejos y diseñar algoritmos eficientes.

1.1.1 Contexto Histórico y Orígenes

Los orígenes de la teoría de grafos se remontan al siglo XVIII, con el famoso problema de los **Puentes de Königsberg** planteado por Leonhard Euler en 1736. La ciudad de Königsberg (actual Kaliningrado) estaba dividida por el río Pregel y contenía siete puentes que conectaban dos islas y las dos orillas del río. El problema consistía en determinar si era posible dar un paseo que cruzara cada uno de los siete puentes exactamente una vez y regresara al punto de partida.

Euler modeló este problema simplificándolo a puntos (las masas de tierra) y líneas (los puentes), sentando las bases de lo que hoy conocemos como grafo. Su conclusión de que tal paseo era imposible no solo resolvió el enigma, sino que también inauguró una nueva rama de las matemáticas.

1.1.2 El Mundo es un Grafo: Ejemplos Intuitivos y Avanzados

Hoy en día, los grafos son omnipresentes y se utilizan para modelar una vasta gama de fenómenos y sistemas:

- **Redes Sociales:** Plataformas como Facebook, Twitter o LinkedIn pueden verse como grafos gigantes. Los **usuarios** son los vértices (nodos) y las **amistades, seguimientos o conexiones** son las aristas. El análisis de estas estructuras permite identificar comunidades, predecir tendencias y optimizar la publicidad.
- **Internet y Redes de Computadoras:** La propia estructura de la World Wide Web puede modelarse como un grafo donde las **páginas web** son nodos y los **hipervínculos** son aristas dirigidas. Las **redes de computadoras** tienen **routers** o **servidores** como vértices y los **cables o conexiones inalámbricas** como aristas.
- **Logística y Transporte:** Sistemas de **navegación** como Google Maps, redes de **transporte público** o rutas de **entrega** utilizan grafos. Las **ciudades o intersecciones** son nodos, y las **calles, carreteras o líneas de metro** son aristas, a menudo con pesos que representan la distancia, el tiempo de viaje o el costo.
- **Sistemas de Recomendación:** Tiendas online como Amazon o servicios de streaming como Netflix usan grafos para conectar **usuarios** con **productos o películas** basándose en interacciones (compras, visualizaciones, valoraciones). Esto permite sugerir nuevos ítems a los usuarios.
- **Bioinformática:** En biología computacional, los grafos se utilizan para modelar **redes de interacción proteica**, **redes genéticas** o para el **ensamblaje de genomas**, donde fragmentos de ADN se conectan para reconstruir una secuencia completa.
- **Ciencia de Datos y Machine Learning:** Los **grafos de conocimiento** representan relaciones semánticas entre entidades. Además, una rama emergente, las **Graph Neural Networks (GNNs)**, extiende las técnicas de aprendizaje profundo a datos estructurados en grafos, abriendo nuevas posibilidades para el análisis de redes complejas.

1.2 Definiciones Fundamentales y Terminología

Para trabajar eficazmente con grafos, es crucial dominar la terminología básica.

1.2.1 Componentes Básicos

Un **grafo** G se define formalmente como un par ordenado de conjuntos $G = (V, E)$, donde: * **Vértices (Nodos):** V es un conjunto finito y no vacío de elementos llamados vértices. Se suelen denotar como $v_1, v_2, \dots, v_n$. * **Aristas (Enlaces, Arcos):** E es un conjunto de pares de vértices, llamados aristas. Las aristas conectan dos vértices y representan una relación entre ellos. Se suelen denotar como $e_1, e_2, \dots, e_m$.

1.2.2 Tipos de Grafos

La naturaleza de las aristas y vértices determina el tipo de grafo:

- **Grafo No Dirigido:** Las aristas no tienen una dirección específica. Si una arista conecta el vértice u con el vértice v , entonces la relación es bidireccional, es decir, (u, v) es lo mismo que (v, u) .

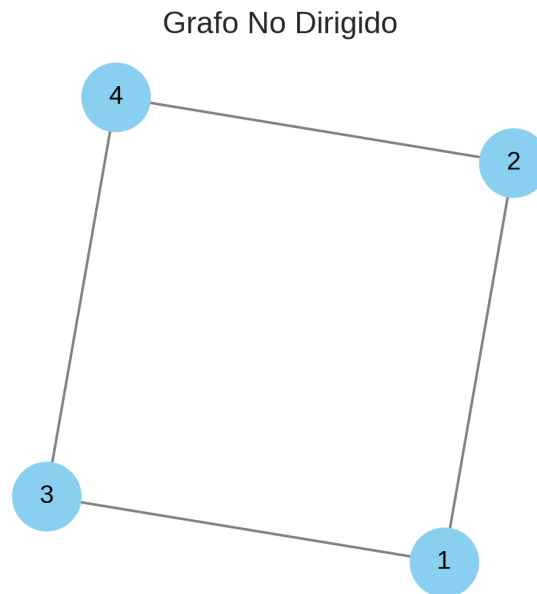


Figura 1.1: Grafo No Dirigido

- **Ejemplo:** Una red de amistades en la que la amistad es mutua.
- **Grafo Dirigido (Dígrafo):** Las aristas tienen una dirección clara, del vértice origen al vértice destino. Una arista $(u \rightarrow v)$ es diferente de una arista $(v \rightarrow u)$.
 - **Ejemplo:** Seguir a alguien en Twitter (el seguimiento no es necesariamente mutuo), una carretera de sentido único.
- **Grafo Ponderado:** Cada arista tiene asociado un valor numérico, llamado **peso**, **costo** o **distancia**, $w(u, v) \in \mathbb{R}$. Estos pesos pueden representar distancias físicas, tiempos de viaje, costos, capacidades, etc.
 - **Ejemplo:** Un mapa de carreteras donde el peso es la distancia entre ciudades.
- **Grafo No Ponderado:** Las aristas no tienen un peso explícito. A menudo, se asume un peso unitario para todas las aristas si se calculan distancias.
 - **Ejemplo:** Un diagrama de flujo donde solo importa la secuencia, no la "costo" de la transición.
- **Grafo Simple:** Un grafo que no contiene lazos (aristas que conectan un vértice consigo mismo, ej., (v, v)) ni aristas múltiples (más de una arista entre el mismo par de vértices).
 - **La mayoría de los algoritmos de grafos asumen grafos simples.**
- **Múltigrafo:** Un grafo que permite la existencia de múltiples aristas entre el mismo par de vértices.
 - **Ejemplo:** Múltiples líneas de autobús que conectan dos paradas.
- **Pseudografo:** Un grafo que permite lazos y aristas múltiples.

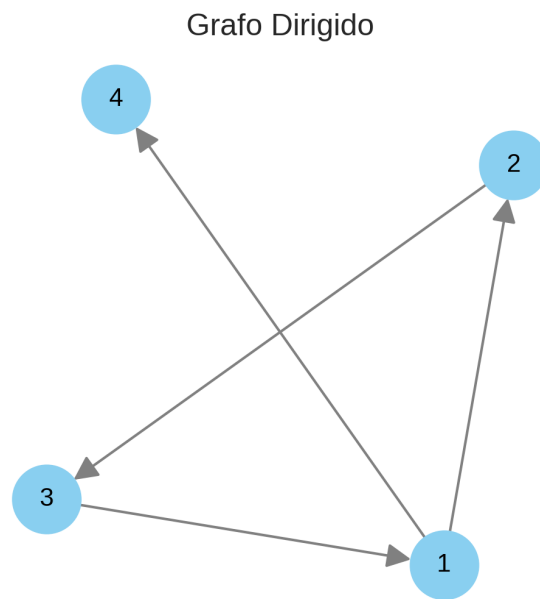


Figura 1.2: Grafo Dirigido

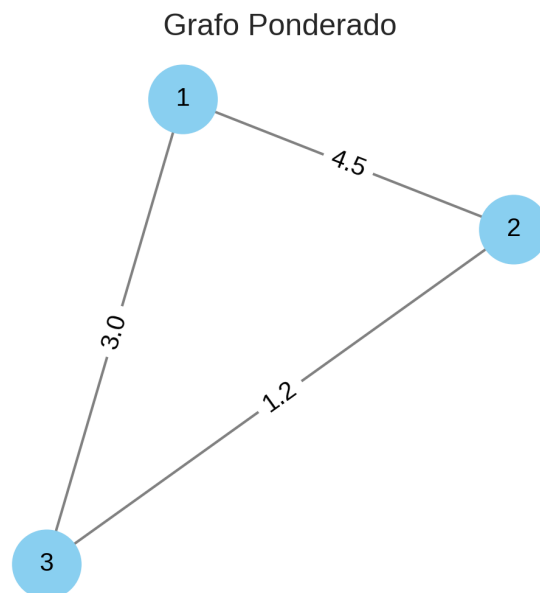


Figura 1.3: Grafo Ponderado

1.2.3 Terminología de Vértices y Aristas

- **Vértices Adyacentes (Vecinos):** Dos vértices u y v son adyacentes si existe una arista que los conecta. En un grafo dirigido, u es adyacente a v si existe una arista $(u \rightarrow v)$.
- **Incidencia:** Una arista es incidente a los vértices que conecta. Por ejemplo, la arista (u, v) es incidente a u y a v .
- **Grado de un Vértice (No Dirigido):** El número de aristas incidentes a un vértice. Se denota como $\text{grado}(v)$. Un lazo contribuye dos veces al grado.
- **Grado de Entrada y Salida (Dirigido):**
 - **Grado de Entrada ($\text{grado}^-(v)$):** Número de aristas que tienen a v como vértice destino.
 - **Grado de Salida ($\text{grado}^+(v)$):** Número de aristas que tienen a v como vértice origen.
- **Vértice Aislado:** Un vértice con grado 0 (no está conectado a ningún otro vértice).
- **Vértice Pendiente (Hoja):** Un vértice con grado 1.
- **Subgrafo:** Un grafo $G' = (V', E')$ es subgrafo de $G = (V, E)$ si $V' \subseteq V$ y $E' \subseteq E$.
 - **Subgrafo Inducido:** Es un subgrafo definido por un subconjunto de vértices $V' \subseteq V$ que incluye *todas* las aristas de E cuyos extremos están en V' .
 - **Supergrafo:** Un grafo G es supergrafo de G' si G' es subgrafo de G .

1.2.4 Caminos y Conectividad

- **Camino (Paseo/Walk):** Una secuencia de vértices $v_0, v_1, \dots, v_k$ tal que cada par (v_i, v_{i+1}) es una arista. En general, se permiten vértices y aristas repetidos.
 - **Camino Simple:** Un camino en el que todos los vértices son distintos.
- **Ciclo:** Un camino cerrado (comienza y termina en el mismo vértice, $v_0 = v_k$) con longitud $k \geq 1$.
 - **Ciclo Simple:** Un ciclo donde todos los vértices son distintos, excepto el primero y el último ($v_0 = v_k$).
- **Longitud de un Camino/Ciclo:**
 - En grafos no ponderados, es el número de aristas en el camino/ciclo.
 - En grafos ponderados, es la suma de los pesos de las aristas en el camino/ciclo.
- **Conectividad (en Grafos No Dirigidos):**
 - **Grafo Conexo:** Si existe al menos un camino entre cada par de vértices en el grafo.
 - **Componente Conexa:** Un subgrafo maximal conexo. Un grafo no conexo se compone de varias componentes conexas.
- **Conectividad (en Grafos Dirigidos):**
 - **Fuertemente Conexo:** Si existe un camino de u a v Y un camino de v a u para *cada* par de vértices (u, v) en el grafo.
 - **Débilmente Conexo:** Si el grafo subyacente (el grafo no dirigido que se obtiene al ignorar las direcciones de las aristas) es conexo.
- **Distancia:** En un grafo ponderado (o no ponderado), la distancia entre dos vértices u y v es la longitud del camino más corto entre ellos.
- **Diámetro:** La mayor de todas las distancias de caminos más cortos entre todos los pares de vértices en el grafo.

1.3 Representaciones Computacionales de Grafos

La elección de cómo representar un grafo en la memoria de una computadora es crucial, ya que afecta directamente la eficiencia (tiempo y espacio) de los algoritmos que se ejecutarán sobre él.

1.3.1 Consideraciones de Diseño

Al elegir una representación, es importante evaluar:

- **Espacio (Complejidad Espacial):** ¿Cuánta memoria utiliza la estructura de datos para almacenar el grafo? Esto a menudo depende del número de vértices (V) y el número de aristas (E).
- **Tiempo de Operaciones Clave (Complejidad Temporal):** La eficiencia de las operaciones más frecuentes:
 - **Agregar/Eliminar un Vértice:** Costo de añadir o quitar un nodo.
 - **Agregar/Eliminar una Arista:** Costo de establecer o romper una conexión.
 - **Consultar si Existe una Arista** (u, v): ¿Es u adyacente a v ?
 - **Obtener Todos los Vecinos de un Vértice** u : ¿Cuáles son los nodos directamente conectados a u ?
 - **Iterar Sobre Todas las Aristas del Grafo:** Costo de recorrer todas las conexiones.
- **Tipos de Grafos (Denso vs. Disperso):**
 - **Grafo Denso:** Un grafo con un número elevado de aristas, cercano al máximo posible ($E \approx V^2$). Para $V = 100$, un grafo denso podría tener miles de aristas.
 - **Grafo Disperso (Sparse):** Un grafo con relativamente pocas aristas ($E \approx V$). Para $V = 100$, un grafo disperso podría tener solo unos cientos de aristas.

1.3.2 Lista de Adyacencia (Adjacency List)

Esta es una de las representaciones más comunes y flexibles.

- **Concepto:** Cada vértice se asocia con una lista (o arreglo) de sus vértices adyacentes. Si el grafo es ponderado, la lista contendrá pares (vecino, peso).
- **Estructuras de Datos en Python:**
 - Para grafos no ponderados, un diccionario donde las claves son vértices y los valores son listas de enteros (vecinos): `dict[int, list[int]]`.
 - Para grafos ponderados, un diccionario donde las claves son vértices y los valores son listas de tuplas (vecino, peso): `dict[int, list[tuple[int, float]]]`.
- **Implementación de Clases en Python:**

```
class GrafoListaAdyacencia:
```

```
    """
```

```
    Representación de un grafo mediante lista de adyacencia.
```

```

Soporta grafos dirigidos/no dirigidos y ponderados/no
ponderados. Los vértices se identifican por enteros de 0 a
n_vertices - 1.
"""
def __init__(self, n_vertices: int, dirigido: bool = False):
    """
    Inicializa un grafo con n_vertices.
    :param n_vertices: Número total de vértices en el grafo.
    :param dirigido: True si el grafo es dirigido, False si es no
                     dirigido.
    """
    self.n = n_vertices
    self.dirigido = dirigido
    # Un diccionario donde cada clave (vértice) mapea a una lista
    # de sus vecinos. Cada vecino es una tupla (ID_vecino,
    # ↪ peso_arista)
    self.adyacencia: dict[int, list[tuple[int, float]]] = \
        {i: [] for i in range(n_vertices)}

def agregar_arista(self, u: int, v: int, peso: float = 1.0):
    """
    Agrega una arista al grafo.
    :param u: Vértice de origen.
    :param v: Vértice de destino.
    :param peso: Peso de la arista (por defecto 1.0 para no
    ↪ ponderados).
    """
    if not (0 <= u < self.n and 0 <= v < self.n):
        raise ValueError(f"Vértices {u} o {v} fuera del rango "
                        f"[0, {self.n-1}]")

    # Para evitar aristas duplicadas si ya existe (u,v)
    if not self.tiene_arista(u, v):
        self.adyacencia[u].append((v, peso))

    if not self.dirigido and not self.tiene_arista(v, u):
        # Añade también (v,u) si no es dirigido
        self.adyacencia[v].append((u, peso))

def obtener_vecinos(self, u: int) -> list[tuple[int, float]]:
    """
    Retorna la lista de tuplas (vecino, peso) para el vértice u.
    :param u: Vértice del cual obtener los vecinos.
    :return: Lista de vecinos y sus pesos.
    """
    if not (0 <= u < self.n):
        raise ValueError(f"Vértice {u} fuera del rango "
                        f"[0, {self.n-1}]")
    return self.adyacencia[u]

```

```

def tiene_arista(self, u: int, v: int) -> bool:
    """
    Verifica si existe una arista entre u y v.
    :param u: Vértice de origen.
    :param v: Vértice de destino.
    :return: True si la arista existe, False en caso contrario.
    """
    if not (0 <= u < self.n and 0 <= v < self.n):
        # 0 lanzar ValueError, dependiendo de la política deseada
        return False
    return any(vecino_id == v for vecino_id, _ in
        ↪ self.adyacencia[u])

def __str__(self):
    s = (f"Grafo (Lista de Adyacencia, "
        f"'{Dirigido' if self.dirigido else 'No Dirigido'}):\n")
    for u in range(self.n):
        s += f"{u}: "
        vecinos_str = [f"({v}, p={p:.2f})"
            for v, p in self.adyacencia[u]]
        s += ", ".join(vecinos_str) + "\n"
    return s

```

• Análisis de Complejidad:

- **Espacio:** $O(V + E)$, donde V es el número de vértices y E es el número de aristas. Para cada vértice, almacenamos su lista de adyacencia, y cada arista se almacena una vez (en grafos dirigidos) o dos veces (en grafos no dirigidos). Esta eficiencia espacial la hace ideal para **grafos dispersos**.
- **Tiempo:**
 - * **Agregar Arista:** $O(1)$ en promedio (si se evita duplicados, podría ser $O(\text{grado}(u))$).
 - * **Consultar Arista** (u, v) : $O(\text{grado}(u))$ en el peor caso (hay que recorrer la lista de adyacencia de u).
 - * **Obtener Vecinos de u :** $O(\text{grado}(u))$.
 - * **Iterar Todas las Aristas:** $O(V + E)$ para recorrer todas las listas de adyacencia.

1.3.3 Matriz de Adyacencia (Adjacency Matrix)

Esta representación es útil en escenarios específicos, principalmente con grafos densos.

- **Concepto:** Un grafo con V vértices se representa mediante una matriz cuadrada $V \times V$.
 - Para grafos no ponderados, $\text{matriz}[i][j]$ es 1 si existe una arista de i a j , y 0 en caso contrario.
 - Para grafos ponderados, $\text{matriz}[i][j]$ almacena el peso de la arista de i a j . Si no hay arista, se puede usar 0, None, o infinito (∞) para indicar su ausencia.
- **Estructuras de Datos en Python:**

- Listas anidadas de Python (list[list[float]]).
- Para mayor eficiencia numérica y de memoria, se prefiere `numpy.ndarray`.

• **Implementación de Clases en Python:**

```
import numpy as np

class GrafoMatrizAdyacencia:
    """
    Representación de un grafo mediante matriz de adyacencia.
    Soporta grafos dirigidos/no dirigidos y ponderados/no
    ponderados. Los vértices se identifican por enteros de 0 a
    n_vertices - 1.
    """
    def __init__(self, n_vertices: int, dirigido: bool = False,
                  valor_no_arista: float = 0.0):
        """
        Inicializa un grafo con n_vertices.
        :param n_vertices: Número total de vértices en el grafo.
        :param dirigido: True si el grafo es dirigido, False si es no
                        dirigido.
        :param valor_no_arista: Valor para indicar la ausencia de una
                               arista (0.0 o float('inf')).
        """
        self.n = n_vertices
        self.dirigido = dirigido
        self.valor_no_arista = valor_no_arista
        # Inicializa la matriz con el valor_no_arista
        self.matriz = np.full((n_vertices, n_vertices),
                              valor_no_arista, dtype=float)

    def agregar_arista(self, u: int, v: int, peso: float = 1.0):
        """
        Agrega una arista al grafo.
        :param u: Vértice de origen.
        :param v: Vértice de destino.
        :param peso: Peso de la arista.
        """
        if not (0 <= u < self.n and 0 <= v < self.n):
            raise ValueError(f"Vértices {u} o {v} fuera del rango "
                             f"[0, {self.n-1}]")

        self.matriz[u][v] = peso
        if not self.dirigido:
            self.matriz[v][u] = peso

    def tiene_arista(self, u: int, v: int) -> bool:
        """
        Verifica si existe una arista entre u y v.
        :param u: Vértice de origen.
        :param v: Vértice de destino.
        :return: True si la arista existe, False en caso contrario.
        """
```

```

    """
    if not (0 <= u < self.n and 0 <= v < self.n):
        return False
    return self.matriz[u][v] != self.valor_no_arista

def obtener_peso_arista(self, u: int, v: int) -> float:
    """
    Retorna el peso de la arista entre u y v.
    :param u: Vértice de origen.
    :param v: Vértice de destino.
    :return: Peso de la arista o valor_no_arista si no existe.
    """
    if not (0 <= u < self.n and 0 <= v < self.n):
        raise ValueError(f"Vértices {u} o {v} fuera del rango "
                        f"[0, {self.n-1}]")
    return self.matriz[u][v]

def obtener_vecinos(self, u: int) -> list[int]:
    """
    Retorna la lista de IDs de los vértices adyacentes a u.
    :param u: Vértice del cual obtener los vecinos.
    :return: Lista de vecinos.
    """
    if not (0 <= u < self.n):
        raise ValueError(f"Vértice {u} fuera del rango "
                        f"[0, {self.n-1}]")
    return [v for v in range(self.n) if self.tiene_arista(u, v)]

def __str__(self):
    s = (f"Grafo (Matriz de Adyacencia, "
        f"'{Dirigido' if self.dirigido else 'No Dirigido'}):\n")
    s += str(self.matriz) + "\n"
    return s

```

• Análisis de Complejidad:

- **Espacio:** $O(V^2)$, ya que la matriz siempre ocupa $V \times V$ posiciones, independientemente del número de aristas. Esto la hace muy eficiente para **grafos densos**, pero derrochadora para grafos dispersos.
- **Tiempo:**
 - * **Agregar Arista:** $O(1)$.
 - * **Consultar Arista** (u, v) : $O(1)$. Esta es su mayor ventaja.
 - * **Obtener Vecinos de u :** $O(V)$ (hay que recorrer la fila completa de u).
 - * **Iterar Todas las Aristas:** $O(V^2)$ (hay que recorrer toda la matriz).

1.3.4 Lista de Aristas (Edge List)

Esta es la representación más sencilla, pero a menudo la menos eficiente para operaciones de grafo generales.

- **Concepto:** El grafo se representa como una lista o colección de todas sus

aristas. Cada arista se almacena como una tupla (u, v, peso) o un objeto Edge.

- **Estructuras de Datos en Python:** list[tuple[int, int, float]] o una lista de instancias de una clase Edge personalizada.
- **Implementación de Clases en Python:**

```
from dataclasses import dataclass

from dataclasses import dataclass

@dataclass
class Arista:
    """
    Clase para representar una arista con origen, destino y peso.
    """
    u: int
    v: int
    peso: float = 1.0

class GrafoListaAristas:
    """
    Representación de un grafo mediante una lista de aristas.
    Soporta grafos dirigidos/no dirigidos y ponderados/no
    ponderados. Los vértices se identifican por enteros de 0 a
    n_vertices - 1.
    """
    def __init__(self, n_vertices: int, dirigido: bool = False):
        """
        Inicializa un grafo con n_vertices.
        :param n_vertices: Número total de vértices en el grafo.
        :param dirigido: True si el grafo es dirigido, False si es no
            dirigido.
        """
        self.n = n_vertices
        self.dirigido = dirigido
        self.aristas: list[Arista] = []
        # Para algunas operaciones eficientes, es útil tener también
        ↪ un
        # set de vértices
        self.vertices: set[int] = set(range(n_vertices))

    def agregar_arista(self, u: int, v: int, peso: float = 1.0):
        """
        Agrega una arista al grafo.
        Para grafos no dirigidos, se añade una arista en cada
        ↪ dirección.
        :param u: Vértice de origen.
        :param v: Vértice de destino.
        :param peso: Peso de la arista.
        """
```

```

if not (0 <= u < self.n and 0 <= v < self.n):
    raise ValueError(f"Vértices {u} o {v} fuera del rango "
                    f"[0, {self.n-1}]")

# Podríamos añadir lógica para evitar duplicados si es
# necesario,
# pero típicamente en esta representación se añaden todas las
# aristas explícitamente.
self.aristas.append(Arista(u, v, peso))
if not self.dirigido:
    self.aristas.append(Arista(v, u, peso))

# Nota: obtener_vecinos y tiene_arista son inherentemente
# ineficientes
# en esta representación si no se mantiene una estructura
# auxiliar.
# Sus complejidades serían  $O(E)$  en el peor caso ya que requieren
# recorrer la lista de aristas.

def __str__(self):
    s = (f"Grafo (Lista de Aristas, "
        f"{'Dirigido' if self.dirigido else 'No Dirigido'}):\\n")
    for arista in self.aristas:
        s += (f"({arista.u} --({arista.peso:.2f})--> "
            f"{arista.v})\\n")
    return s

```

• Análisis de Complejidad:

- **Espacio:** $O(E)$, ya que solo se almacenan las aristas.
- **Tiempo:**
 - * **Agregar Arista:** $O(1)$.
 - * **Consultar Arista** (u, v) : $O(E)$ (hay que recorrer toda la lista).
 - * **Obtener Vecinos de u :** $O(E)$ (hay que recorrer toda la lista para encontrar aristas incidentes a u).
 - * **Iterar Todas las Aristas:** $O(E)$.
- **Ventaja:** Simplicidad. Es particularmente útil para algoritmos que procesan *todas* las aristas del grafo de forma independiente o que requieren ordenar las aristas (como el algoritmo de Kruskal para MST).

1.4 Comparación y Elección de Representaciones

La elección de la representación adecuada del grafo es un primer paso crítico en el diseño de cualquier algoritmo de grafos eficiente.

1.4.1 Tabla Comparativa Detallada

Operación / Característica	Lista de Adyacencia	Matriz de Adyacencia	Lista de Aristas
Espacio	$O(V + E)$	$O(V^2)$	$O(E)$
Agregar Arista	$O(1)$	$O(1)$	$O(1)$
Consultar Arista (u, v)	$O(\text{grado}(u))$	$O(1)$	$O(E)$
Obtener Vecinos de u	$O(\text{grado}(u))$	$O(V)$	$O(E)$
Iterar Todas las Aristas	$O(V + E)$	$O(V^2)$	$O(E)$
Adecuado para Grafos	Dispersos	Densos	Ciertos algoritmos (ej. Kruskal)

1.4.2 Casos de Uso

- **Lista de Adyacencia:**
 - Es la representación **predominante** para la mayoría de los algoritmos de grafos, especialmente aquellos basados en recorridos (BFS, DFS) y problemas de caminos más cortos (Dijkstra, Bellman-Ford).
 - Excelente para **grafos dispersos** donde $E \ll V^2$, ya que su consumo de memoria es proporcional al número real de conexiones.
 - Cuando las operaciones más frecuentes son “obtener los vecinos de un vértice” o “recorrer el grafo”.
- **Matriz de Adyacencia:**
 - Ideal para **grafos densos** donde $E \approx V^2$.
 - Cuando se necesita una **consulta de existencia de aristas extremadamente rápida** ($O(1)$).
 - Útil en algoritmos que implican iterar sobre todos los pares de vértices o en algoritmos de programación dinámica como Floyd-Warshall.
 - Su desventaja es el alto consumo de memoria $O(V^2)$, lo que la hace impráctica para grafos con muchos vértices.
- **Lista de Aristas:**
 - Menos versátil como representación general.
 - Principalmente útil para algoritmos que necesitan **procesar todas las aristas de forma independiente**, como el algoritmo de Kruskal para el Árbol de Expansión Mínima (donde las aristas se ordenan por peso).
 - También es conveniente cuando la definición del grafo se basa puramente en la colección de sus conexiones.

1.4.3 Librerías de Grafos en Python (Introducción)

Si bien entender las representaciones subyacentes es vital, en la práctica, los desarrolladores de Python a menudo recurren a librerías maduras que abstraen estas complejidades y ofrecen una API de alto nivel. La más popular es **NetworkX**:

- **NetworkX:** Una potente librería de Python para la creación, manipulación y estudio de la estructura, dinámica y funciones de redes complejas. Permite crear grafos de manera sencilla, añadir nodos y aristas, y proporciona implementaciones de la mayoría de los algoritmos de grafos. Utiliza internamente una variante de lista de adyacencia (diccionario de diccionarios) y ofrece métodos muy optimizados.

```
import networkx as nx

# Crear un grafo no dirigido
G = nx.Graph()
G.add_nodes_from([0, 1, 2, 3])
G.add_edge(0, 1, weight=1.0)
G.add_edge(0, 2, weight=2.0)
G.add_edge(1, 3, weight=3.0)
G.add_edge(2, 3, weight=1.5)

print("Grafo NetworkX (no dirigido):")
# Mostrar aristas con sus datos (pesos)
print(list(G.edges(data=True)))

# Crear un grafo dirigido
D = nx.DiGraph()
D.add_edges_from([(0, 1), (0, 2), (1, 3), (2, 3)])
print("\nGrafo NetworkX (dirigido):")
print(list(D.edges()))

# Operaciones básicas
print(f"\nVecinos de 0 en G: {list(G.neighbors(0))}")
print(f"Existe arista (0,1) en G: {G.has_edge(0, 1)}")
print(f"Peso de arista (0,2) en G: {G[0][2]['weight']}")
```

Entender las representaciones manuales es clave para comprender cómo funcionan las librerías internamente, para depurar problemas de rendimiento, y para implementar algoritmos personalizados o variantes no cubiertas por las librerías estándar.

Capítulo 2

Recorridos y Búsqueda en Grafos

Este capítulo explora los algoritmos fundamentales para explorar sistemáticamente los vértices y aristas de un grafo: la Búsqueda en Amplitud (BFS) y la Búsqueda en Profundidad (DFS). Ambos son pilares para una amplia gama de problemas y algoritmos más complejos en la teoría de grafos.

2.1 2.1. Introducción a los Recorridos en Grafos

2.1.1 2.1.1. ¿Qué es un Recorrido?

Un **recorrido en un grafo** es el proceso de visitar (examinar o procesar) sistemáticamente cada vértice y cada arista de un grafo de una manera estructurada. No se trata simplemente de pasar por los nodos, sino de explorar las relaciones entre ellos para obtener información sobre la estructura del grafo.

2.1.2 2.1.2. Aplicaciones Fundamentales

Los algoritmos de recorrido son la base para resolver numerosos problemas, incluyendo:

- **Verificar Conectividad:** Determinar si un grafo es conexo o fuertemente conexo.
 - **Encontrar Caminos:** Descubrir si existe una ruta entre dos vértices y, en algunos casos, encontrar el camino más corto o más largo.
 - **Detectar Ciclos:** Identificar la presencia de ciclos en un grafo, crucial para DAGs.
 - **Construir Árboles de Expansión:** Generar un subgrafo que es un árbol y conecta todos los vértices.
 - **Determinar la Estructura del Grafo:** Comprender la topología, la densidad o la distribución de componentes.
-

2.2. Búsqueda en Amplitud (BFS - Breadth-First Search)

BFS es un algoritmo de recorrido de grafos que explora el grafo “capa por capa”, visitando primero todos los vecinos directos de un nodo antes de pasar a los vecinos de esos vecinos.

2.2.1. Concepto y Funcionamiento

Imaginen que están tirando una piedra en un estanque. Las ondas se expanden concéntricamente desde el punto de impacto. BFS funciona de manera similar:

1. Comienza en un **vértice inicial (fuente)**.
2. Visita a **todos sus vecinos directos** (distancia 1).
3. Luego visita a **todos los vecinos de esos vecinos** (distancia 2), y así sucesivamente.

BFS: Exploración por Niveles

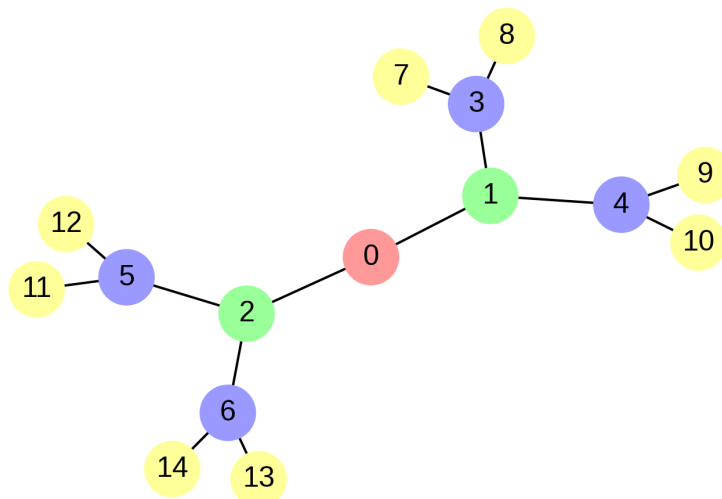


Figura 2.1: Exploración por Niveles en BFS

Para lograr esta expansión por capas, BFS utiliza una **cola (FIFO - First-In, First-Out)**. Los vértices se añaden a la cola y se procesan en el orden en que fueron descubiertos.

2.2.2. Algoritmo Detallado

1. Inicialización:

- Crear una lista o conjunto visitados para mantener un registro de los vértices ya explorados (inicialmente, todos no están visitados).
- Crear una cola (usando `collections.deque` en Python).
- Marcar el `vértice_inicial` como visitado y añadirlo a la cola.
- Crear una lista `orden_recorrido` para almacenar el orden de visita.

2. Exploración (mientras la cola no esté vacía):

- **Desencolar** un vértice `u` de la parte frontal de la cola.
- Añadir `u` a `orden_recorrido`.

- Para **cada vecino v** de u:
 - Si v **no ha sido visitado**:
 - * Marcar v como visitado.
 - * **Encolar** v en la parte trasera de la cola.

2.2.3 Implementación en Python

Para una implementación eficiente de la cola en Python, se recomienda `collections.deque`.

```
from collections import deque
from typing import List, Tuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1
# class GrafoListaAdyacencia:
#     def __init__(self, n_v: int, dirigido: bool = False): ...
#     def agregar_arista(self, u: int, v: int, p: float = 1.0): ...
#     def obtener_vecinos(self, u: int) -> List[Tuple[int, float]]: ...

def bfs(grafo, inicio: int) -> List[int]:
    """
    Realiza un recorrido BFS desde un vértice de inicio dado.
    :param grafo: Una instancia de GrafoListaAdyacencia.
    :param inicio: El vértice de inicio (entero).
    :return: Una lista de vértices en el orden BFS.
    """
    n = grafo.n
    visitados = [False] * n
    cola = deque()
    orden_recorrido = []

    if not (0 <= inicio < n):
        raise ValueError(f"Inicio {inicio} fuera del rango "
                        f"[0, {n-1}]")

    visitados[inicio] = True
    cola.append(inicio)

    while cola:
        u = cola.popleft()
        orden_recorrido.append(u)

        for v, _ in grafo.obtener_vecinos(u):
            if not visitados[v]:
                visitados[v] = True
                cola.append(v)

    return orden_recorrido

# Ejemplo de uso (descomentar para probar si se tiene la clase Grafo)
# g_no_dirigido = GrafoListaAdyacencia(5, dirigido=False)
# g_no_dirigido.agregar_arista(0, 1)
# g_no_dirigido.agregar_arista(0, 2)
```

```
# g_no_dirigido.agregar_arista(1, 3)
# g_no_dirigido.agregar_arista(2, 4)
# print(f"BFS desde 0: {bfs(g_no_dirigido, 0)}")
```

2.2.4 2.2.4. Análisis de Complejidad

- **Tiempo:** $O(V + E)$, donde V es el número de vértices y E el número de aristas. Cada vértice se encola y desencola una vez, y para cada vértice, se recorren sus aristas una vez.
- **Espacio:** $O(V)$ en el peor caso, para almacenar la lista visitados y los vértices en la cola.

2.2.5 2.2.5. Aplicaciones de BFS

- **Caminos Más Cortos en Grafos No Ponderados:** BFS encuentra el camino más corto (en número de aristas) desde el inicio a todos los demás vértices.
- **Detección de Componentes Conexas:** Múltiples llamadas a BFS (desde vértices no visitados) identifican las componentes conexas.
- **Verificación de Bipartición (2-Coloreado):** BFS puede 2-colorear un grafo; si falla, no es bipartito.
- **Nivelado de Redes (Layering):** Determina la distancia mínima (en aristas) desde un nodo fuente.

2.3 2.3. Búsqueda en Profundidad (DFS - Depth-First Search)

DFS es un algoritmo de recorrido que explora tan profundo como sea posible a lo largo de cada rama antes de retroceder.

2.3.1 2.3.1. Concepto y Funcionamiento

Imaginen que están explorando un laberinto con un trozo de cuerda. Siguen un camino hasta el final (o hasta que chocan con una pared o un camino ya explorado), luego retroceden un poco y toman un nuevo giro.

DFS funciona siguiendo un camino desde el nodo de inicio hasta que no puede avanzar más, marcando los nodos visitados en el camino. Una vez que llega a un “callejón sin salida” (un nodo sin vecinos no visitados), retrocede al nodo anterior y explora otra rama, utilizando una **pila (LIFO - Last-In, First-Out)** o la **pila de llamadas de función** en el caso de implementaciones recursivas.

2.3.2 2.3.2. Algoritmo Detallado

Versión Recursiva:

1. Marcar el vértice actual u como visitado.
2. Añadir u al orden_recorrido.
3. Para **cada vecino** v de u :
 - Si v **no ha sido visitado**:

- Llamar recursivamente a `dfs(grafo, v, visitados, orden_recorrido)`.

Versión Iterativa (usando una pila explícita):

1. Inicialización:

- Crear una lista o conjunto visitados.
- Crear una pila (lista de Python).
- Añadir el vértice_inicial a la pila.
- Crear una lista orden_recorrido.

2. Exploración (mientras la pila no esté vacía):

- **Desapilar** un vértice `u` de la cima de la pila.
- Si `u` **no ha sido visitado**:
 - Marcar `u` como visitado.
 - Añadir `u` a `orden_recorrido`.
 - Para **cada vecino `v`** de `u`:
 - * Si `v` **no ha sido visitado**:
 - **Apilar** `v` en la pila. (El orden de apilado puede afectar el `orden_recorrido` final, pero no la validez de la exploración).

2.3.3 2.3.3. Implementación en Python

2.3.3.1 Versión Recursiva

Esta es la forma más intuitiva y común de implementar DFS.

```
from typing import List, Tuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1

def dfs_recursivo(grafo, u: int, visitados: List[bool],
                  orden_recorrido: List[int]):
    """
    Función auxiliar recursiva para DFS.
    :param grafo: Instancia de GrafoListaAdyacencia.
    :param u: Vértice actual.
    :param visitados: Lista booleana para rastrear vértices visitados.
    :param orden_recorrido: Lista para acumular el orden de visita.
    """
    visitados[u] = True
    orden_recorrido.append(u)

    for v, _ in grafo.obtener_vecinos(u):
        if not visitados[v]:
            dfs_recursivo(grafo, v, visitados, orden_recorrido)

def dfs(grafo, inicio: int) -> List[int]:
    """
    Realiza un recorrido DFS desde un vértice de inicio dado.
    :param grafo: Instancia de GrafoListaAdyacencia.
    :param inicio: El vértice de inicio (entero).
    :return: Una lista de vértices en el orden DFS.
    """
```

```

n = grafo.n
visitados = [False] * n
orden_recorrido = []

if not (0 <= inicio < n):
    raise ValueError(f"Inicio {inicio} fuera del rango "
                     f"[0, {n-1}]")

dfs_recursoivo(grafo, inicio, visitados, orden_recorrido)
return orden_recorrido

# Ejemplo de uso (descomentar para probar si se tiene la clase Grafo)
# g_no_dirigido = GrafoListaAdyacencia(5, dirigido=False)
# g_no_dirigido.agregar_arista(0, 1)
# g_no_dirigido.agregar_arista(0, 2)
# g_no_dirigido.agregar_arista(1, 3)
# g_no_dirigido.agregar_arista(2, 4)
# print(f"DFS desde 0: {dfs(g_no_dirigido, 0)}")
# Salida: [0, 1, 3, 2, 4] o similar

```

2.3.3.2 Versión Iterativa

Útil para evitar límites de recursión en Python para grafos muy grandes.

```

from typing import List, Tuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1

def dfs_iterativo(grafo, inicio: int) -> List[int]:
    """
    Realiza un recorrido DFS iterativo desde un vértice de inicio dado.
    :param grafo: Instancia de GrafoListaAdyacencia.
    :param inicio: El vértice de inicio (entero).
    :return: Una lista de vértices en el orden DFS.
    """
    n = grafo.n
    visitados = [False] * n
    pila = []
    orden_recorrido = []

    if not (0 <= inicio < n):
        raise ValueError(f"Inicio {inicio} fuera del rango "
                         f"[0, {n-1}]")

    pila.append(inicio)
    while pila:
        u = pila.pop() # LIFO
        if not visitados[u]:
            visitados[u] = True
            orden_recorrido.append(u)

```

```
# Empilar vecinos en orden inverso para que el primer vecino
# en la lista de adyacencia se procese primero
↪ (comportamiento DFS)
# Nota: el orden exacto de los vecinos puede influir en el
# orden_recorrido final si el grafo permite múltiples caminos.
for v, _ in reversed(grafo.obtener_vecinos(u)):
    if not visitados[v]:
        pila.append(v)
return orden_recorrido
```

2.3.4 2.3.4. Análisis de Complejidad

- **Tiempo:** $O(V + E)$, similar a BFS, ya que cada vértice y arista se visita a lo sumo una vez.
- **Espacio:**
 - **Versión Recursiva:** $O(V)$ en el peor caso, limitado por la profundidad de la recursión.
 - **Versión Iterativa:** $O(V)$ si se controla la duplicidad en la pila; sin embargo, implementaciones sencillas (como la mostrada arriba) pueden crecer hasta $O(E)$ en grafos densos al almacenar múltiples referencias al mismo vértice antes de visitarlo.

2.3.5 2.3.5. Aplicaciones de DFS

- **Detección de Ciclos:** En un grafo dirigido, DFS detecta ciclos al encontrar una arista de retroceso a un vértice ya en la pila de recursión.
 - **Ordenamiento Topológico:** Para DAGs, el orden de finalización de DFS es el inverso de un ordenamiento topológico.
 - **Componentes Fuertemente Conexas (SCCs):** DFS es la base de algoritmos como Tarjan o Kosaraju para SCCs.
 - **Resolución de Laberintos:** DFS explora un camino hasta el final, luego retrocede y prueba otra rama.
 - **Búsqueda de Caminos:** Puede encontrar un camino entre dos vértices, no necesariamente el más corto.
 - **Conectividad:** Determina si un grafo es conexo (si todos los vértices son alcanzables desde un inicio).
-

Capítulo 3

Camino Más Corto: Algoritmos Clásicos

Este capítulo se adentra en uno de los problemas fundamentales de la teoría de grafos: encontrar el camino más corto entre dos vértices. Exploraremos algoritmos clásicos y eficientes como Dijkstra, Bellman-Ford y Floyd-Warshall, entendiendo sus principios, implementaciones en Python y las condiciones bajo las cuales cada uno es más adecuado.

3.1 Introducción al Problema del Camino Más Corto

3.1.1 Definición del Problema

El **problema del camino más corto** consiste en encontrar una ruta entre dos vértices (o entre un vértice y todos los demás) en un grafo, tal que la suma de los pesos de las aristas a lo largo de esa ruta sea mínima.

- **Camino Más Corto de Fuente Única (Single-Source Shortest Path - SSSP):** Encontrar el camino más corto desde un vértice s a todos los demás vértices en el grafo.
- **Camino Más Corto de Todos los Pares (All-Pairs Shortest Path - APSP):** Encontrar los caminos más cortos entre cada par de vértices en el grafo.

3.1.2 Tipos de Grafos y Consideraciones

La elección del algoritmo depende crucialmente del tipo de grafo: * **Grafos No Ponderados:** BFS puede encontrar los caminos más cortos (en número de aristas). * **Grafos Ponderados con Pesos No Negativos:** Algoritmo de Dijkstra. * **Grafos Ponderados con Pesos Negativos (sin ciclos negativos):** Algoritmo de Bellman-Ford. * **Grafos con Ciclos Negativos:** No existe un camino más corto bien definido (el camino puede ser infinitamente “corto” al recorrer el ciclo). Algunos algoritmos pueden detectarlos.

3.2 Algoritmo de Dijkstra

Dijkstra es el algoritmo más conocido para encontrar los caminos más cortos desde un vértice fuente a todos los demás vértices en un grafo con aristas de pesos **no negativos**.

3.2.1 Concepto y Funcionamiento

Dijkstra funciona como una “expansión de ondas”, similar a BFS pero priorizando los vértices con la distancia más pequeña conocida hasta el momento. Mantiene y actualiza las distancias más cortas estimadas a cada vértice.

1. Inicialización:

- Asigna una distancia de 0 al vértice fuente y infinito a todos los demás.
- Mantiene un conjunto de vértices “visitados” (o “finalizados”).
- Utiliza una **cola de prioridad** (min-heap) para seleccionar eficientemente el vértice no visitado con la distancia más pequeña.

2. Iteración:

- Extrae el vértice u con la distancia mínima de la cola de prioridad.
- Marca u como visitado.
- Para cada vecino v de u :
 - **Relajación:** Si la distancia a u más el peso de la arista (u, v) es menor que la distancia actual a v , actualiza la distancia a v y (potencialmente) actualiza su posición en la cola de prioridad.

3.2.2 Algoritmo Detallado

1. $\text{dist}[v] = \text{infinity}$ para todos los v , $\text{dist}[s] = 0$.
2. $\text{prev}[v] = \text{undefined}$ para todos los v .
3. $\text{min_heap} = [(0, s)]$ (distancia, vértice).
4. Mientras min_heap no esté vacía:
 - a. $d_u, u = \text{min_heap.pop_min}()$ (extrae el vértice u con la d_u más pequeña).
 - b. Si $d_u > \text{dist}[u]$, continuar (ya hemos encontrado un camino más corto a u).
 - c. Para cada arista (u, v) con peso w :
 - i. Si $\text{dist}[u] + w < \text{dist}[v]$:
 - $\text{dist}[v] = \text{dist}[u] + w$.
 - $\text{prev}[v] = u$.
 - $\text{min_heap.push}((\text{dist}[v], v))$.

3.2.3 Implementación en Python

Se utiliza `heapq` para la cola de prioridad.

```
import heapq
from collections import deque
from typing import List, Tuple, Dict, Optional
```

```
# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1
# y sus vecinos retornan (id_vecino, peso)
```

```

def dijkstra(grafo, inicio: int) -> Tuple[List[float],
↳ List[Optional[int]]]:
    """
    Calcula los caminos más cortos desde un vértice de inicio a todos
    los demás vértices en un grafo ponderado con pesos no negativos.
    :param grafo: Una instancia de GrafoListaAdyacencia.
    :param inicio: El vértice de inicio (entero).
    :return: Una tupla (distancias, predecesores).
               distancias[v] es la distancia más corta desde inicio a v.
               predecesores[v] es el vértice anterior a v en el camino más
↳ corto.
    """
    n = grafo.n
    distancias = [float('inf')] * n
    predecesores: List[Optional[int]] = [None] * n
    distancias[inicio] = 0

    # Cola de prioridad: (distancia_actual, vertice)
    cola_prioridad: List[Tuple[float, int]] = [(0, inicio)]

    while cola_prioridad:
        dist_u, u = heapq.heappop(cola_prioridad)

        # Si ya hemos encontrado un camino más corto a u
        # (por una entrada anterior en la cola de prioridad)
        if dist_u > distancias[u]:
            continue

        for v, peso_uv in grafo.obtener_vecinos(u):
            if distancias[u] + peso_uv < distancias[v]:
                distancias[v] = distancias[u] + peso_uv
                predecesores[v] = u
                heapq.heappush(cola_prioridad, (distancias[v], v))

    return distancias, predecesores

# Función para reconstruir el camino
def reconstruir_camino(predecesores: List[Optional[int]], inicio: int,
    destino: int) -> List[int]:
    """
    Reconstruye el camino más corto desde el origen hasta el destino
    usando la lista de predecesores.
    :param predecesores: Lista de predecesores generada por
↳ Dijkstra/Bellman-Ford.
    :param inicio: El vértice de origen (para validación).
    :param destino: El vértice destino.
    :return: Una lista de vértices que forman el camino más corto.
    """
    camino = deque()
    actual = destino

```

```

while actual is not None:
    camino.appendleft(actual)
    if actual == inicio:
        break
    actual = predecesores[actual]

# Si el camino está vacío o el primer nodo no es el inicio,
# significa que no hay conexión
if not camino or camino[0] != inicio:
    return []

return list(camino)

# Ejemplo de uso (asumiendo GrafoListaAdyacencia del Cap 1)
# g_pond = GrafoListaAdyacencia(5) # No dirigido por defecto
# g_pond.agregar_arista(0, 1, 10)
# g_pond.agregar_arista(0, 2, 3)
# g_pond.agregar_arista(1, 2, 1)
# g_pond.agregar_arista(1, 3, 2)
# g_pond.agregar_arista(2, 1, 4)
# g_pond.agregar_arista(2, 3, 8)
# g_pond.agregar_arista(2, 4, 2)
# g_pond.agregar_arista(3, 4, 5)

# dists, prevs = dijkstra(g_pond, 0)
# print(f"Distancias desde 0: {dists}") # [0, 4, 3, 6, 5]
# print(f"Camino a 4: {reconstruir_camino(prevs, 0, 4)}") # [0, 2, 4]

```

3.2.4 Análisis de Complejidad

- **Tiempo:** $O(E \log V)$ si se usa una cola de prioridad basada en min-heap binario (como `heapq` de Python). Cada `heappush` y `heappop` cuesta $O(\log K)$ donde K es el tamaño del heap.
- **Espacio:** $O(V + E)$ para almacenar distancias, predecesores y la cola de prioridad.

3.2.5 Limitaciones

- **No funciona con aristas de pesos negativos.** Si existen, puede dar resultados incorrectos.
- En el peor caso, puede visitar múltiples veces el mismo nodo si su distancia se actualiza a un valor menor, lo que se gestiona eficientemente con la cola de prioridad.

3.3 Algoritmo de Bellman-Ford

Bellman-Ford resuelve el problema del camino más corto de fuente única en grafos que pueden contener aristas con pesos **negativos**, siempre y cuando no haya **ciclos negativos** alcanzables desde la fuente. También puede detectar la presencia de tales ciclos negativos.

3.3.1 Concepto y Funcionamiento

A diferencia de Dijkstra, Bellman-Ford no es “codicioso”. En lugar de elegir siempre el vértice más cercano, relaja sistemáticamente *todas* las aristas del grafo varias veces. Se basa en la observación de que, en un camino más corto de k aristas, la distancia a un vértice solo puede provenir de un vértice alcanzado con $k - 1$ aristas.

1. **Inicialización:** Similar a Dijkstra, asigna distancia 0 a la fuente y infinito a los demás.
2. **Relajación en Fases:**
 - Itera $V - 1$ veces (donde V es el número de vértices).
 - En cada iteración, examina *todas* las aristas del grafo.
 - Para cada arista (u, v) con peso w , intenta relajar la distancia a v usando la distancia a u .
3. **Detección de Ciclos Negativos:** Después de $V - 1$ iteraciones, si se puede realizar *otra* relajación en cualquier arista en la V -ésima iteración, significa que hay un ciclo negativo en el grafo (alcanzable desde la fuente).

3.3.2 Algoritmo Detallado

1. $\text{dist}[v] = \text{infinity}$ para todos los v , $\text{dist}[s] = 0$.
2. $\text{prev}[v] = \text{undefined}$ para todos los v .
3. Para i de 1 a $V - 1$:
 - a. Para cada arista (u, v) con peso w en el grafo:
 - i. Si $\text{dist}[u] + w < \text{dist}[v]$:
 - $\text{dist}[v] = \text{dist}[u] + w$.
 - $\text{prev}[v] = u$.
4. **Verificación de Ciclos Negativos:**
 - a. Para cada arista (u, v) con peso w en el grafo:
 - i. Si $\text{dist}[u] + w < \text{dist}[v]$:
 - Retorna un error o flag indicando la presencia de un ciclo negativo.
5. Retorna dist y prev .

3.3.3 Implementación en Python

```
from typing import List, Tuple, Optional, NamedTuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1
# Y que ahora tiene un método para obtener todas las aristas si se usa
# una lista de adyacencia (o se podría construir una lista de aristas)

class Edge(NamedTuple):
```

```

u: int
v: int
peso: float

def bellman_ford(grafo, inicio: int) \
    -> Tuple[List[float], List[Optional[int]], bool]:
    """
    Calcula los caminos más cortos desde un vértice de inicio a todos
    los demás, y detecta ciclos negativos.
    :param grafo: Una instancia de GrafoListaAdyacencia.
    :param inicio: El vértice de inicio (entero).
    :return: Tupla (distancias, predecesores, hay_ciclo_negativo).
             Si hay_ciclo_negativo es True, distancias y predecesores
             pueden no ser válidos.
    """
    n = grafo.n
    distancias = [float('inf')] * n
    predecesores: List[Optional[int]] = [None] * n
    distancias[inicio] = 0

    # Construir una lista de todas las aristas para iterar fácilmente
    todas_las_aristas: List[Edge] = []
    for u_node in range(n):
        for v_node, peso in grafo.obtener_vecinos(u_node):
            todas_las_aristas.append(Edge(u_node, v_node, peso))

    # Relajación V-1 veces
    for _ in range(n - 1):
        for edge in todas_las_aristas:
            u, v, peso_uv = edge
            if distancias[u] != float('inf') \
                and distancias[u] + peso_uv < distancias[v]:
                distancias[v] = distancias[u] + peso_uv
                predecesores[v] = u

    # Detectar ciclos negativos (V-ésima relajación)
    hay_ciclo_negativo = False
    for edge in todas_las_aristas:
        u, v, peso_uv = edge
        if distancias[u] != float('inf') \
            and distancias[u] + peso_uv < distancias[v]:
            hay_ciclo_negativo = True
            break # Ciclo negativo encontrado

    return distancias, predecesores, hay_ciclo_negativo

# Ejemplo de uso (asumiendo GrafoListaAdyacencia del Cap 1)
# g_neg = GrafoListaAdyacencia(4, dirigido=True)
# g_neg.agregar_arista(0, 1, 1)
# g_neg.agregar_arista(0, 2, 4)

```

```

# g_neg.agregar_arista(1, 2, -3)
# g_neg.agregar_arista(1, 3, 2)
# g_neg.agregar_arista(2, 3, 3)

# dists, prevs, neg_cycle = bellman_ford(g_neg, 0)
# if neg_cycle:
#     print("Ciclo negativo detectado!")
# else:
#     print(f"Distancias desde 0: {dists}") # [0, 1, -2, 1]
#     print(f"Camino a 3: {reconstruir_camino(prevs, 0, 3)}") # [0, 1, 2,
↪ 3]

# # Ejemplo con ciclo negativo
# g_neg_cycle = GrafoListaAdyacencia(3, dirigido=True)
# g_neg_cycle.agregar_arista(0, 1, 1)
# g_neg_cycle.agregar_arista(1, 2, -1)
# g_neg_cycle.agregar_arista(2, 0, -1) # Ciclo negativo 0 -> 1 -> 2 -> 0
↪ (-1)
# dists, prevs, neg_cycle = bellman_ford(g_neg_cycle, 0)
# if neg_cycle:
#     print("Ciclo negativo detectado!") # Esto debería imprimirse

```

3.3.4 Análisis de Complejidad

- **Tiempo:** $O(V \cdot E)$, ya que se itera $V - 1$ veces y en cada iteración se recorren todas las E aristas.
- **Espacio:** $O(V + E)$ para almacenar distancias, predecesores y la lista de aristas.

3.3.5 Ventajas y Desventajas

- **Ventaja:** Maneja aristas con pesos negativos y detecta ciclos negativos.
- **Desventaja:** Mucho más lento que Dijkstra para grafos sin pesos negativos.

3.4 Algoritmo de Floyd-Warshall

Floyd-Warshall es un algoritmo de programación dinámica que encuentra los caminos más cortos **entre todos los pares de vértices** en un grafo ponderado (con o sin pesos negativos, pero sin ciclos negativos).

3.4.1 Concepto y Funcionamiento

El algoritmo construye progresivamente una matriz de distancias. La idea central es considerar, en cada paso, un subconjunto de vértices intermedios (k) por los cuales los caminos pueden pasar.

1. Inicialización:

- Una matriz de distancias $D[i][j]$ se inicializa con el peso directo de la arista (i, j) si existe, 0 si $i = j$, e infinito si no hay arista directa.
2. **Iteración Principal:**
 - Para cada vértice k (de 0 a $V - 1$):
 - Considera a k como un posible vértice intermedio en todos los caminos.
 - Para cada par de vértices (i, j) :
 - * $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$. Esta operación intenta mejorar el camino de i a j pasando por k .
 3. **Detección de Ciclos Negativos:** Si después de todas las iteraciones, $D[i][i]$ es negativo para cualquier i , existe un ciclo negativo en el grafo.

3.4.2 Algoritmo Detallado

1. Crear una matriz $\text{dist}[V][V]$.
2. Inicializar $\text{dist}[i][j]$ como:
 - $\text{peso}(i, j)$ si existe arista.
 - 0 si $i == j$.
 - infinito si no hay arista.
3. Para k de 0 a $V - 1$: (vértice intermedio)
 - a. Para i de 0 a $V - 1$: (vértice de origen)
 - i. Para j de 0 a $V - 1$: (vértice de destino)
 - $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
4. **Detección de Ciclos Negativos:** Después de las 3 bucles, si $\text{dist}[i][i] < 0$ para cualquier i , hay un ciclo negativo.

3.4.3 Implementación en Python

```
import numpy as np
from typing import List, Tuple

# Asumimos que GrafoMatrizAdyacencia está definido como en el Capítulo 1
# o se puede construir una matriz de adyacencia/distancia inicial.

def floyd_warshall(grafo) -> Tuple[np.ndarray, bool]:
    """
    Calcula los caminos más cortos entre todos los pares de vértices.
    Detecta ciclos negativos.
    :param grafo: Una instancia de GrafoListaAdyacencia o
    ↪ GrafoMatrizAdyacencia.
                   Si es ListaAdyacencia, se construye la matriz inicial
    ↪ aquí.
    :return: Tupla (matriz_distancias, hay_ciclo_negativo).
              matriz_distancias[i][j] es la distancia más corta de i a j.
    """
    n = grafo.n
    # Inicializar matriz de distancias
    # numpy.full para eficiencia, float('inf') para aristas ausentes
    dist = np.full((n, n), float('inf'), dtype=float)

    # Distancia de un vértice a sí mismo es 0
```

```

for i in range(n):
    dist[i][i] = 0

# Llenar con pesos de aristas directas
if hasattr(grafo, 'matriz'): # Si es GrafoMatrizAdyacencia
    # Asegurarse de no sobrescribir INF con 0 si es grafo no ponderado
    for i in range(n):
        for j in range(n):
            if grafo.matriz[i][j] != grafo.valor_no_arista:
                dist[i][j] = grafo.matriz[i][j]
else: # Si es GrafoListaAdyacencia
    for u in range(n):
        for v, peso in grafo.obtener_vecinos(u):
            dist[u][v] = peso

# Algoritmo principal de Floyd-Warshall
for k in range(n): # Vértice intermedio
    for i in range(n): # Vértice de origen
        for j in range(n): # Vértice de destino
            if dist[i][k] != float('inf') \
                and dist[k][j] != float('inf') \
                and dist[i][k] + dist[k][j] < dist[i][j]:
                dist[i][j] = dist[i][k] + dist[k][j]

# Detectar ciclos negativos
hay_ciclo_negativo = False
for i in range(n):
    if dist[i][i] < 0:
        hay_ciclo_negativo = True
        break

return dist, hay_ciclo_negativo

# Ejemplo de uso (asumiendo GrafoListaAdyacencia o Matriz)
# g_apsp = GrafoListaAdyacencia(4, dirigido=True)
# g_apsp.agregar_arista(0, 1, 3)
# g_apsp.agregar_arista(0, 3, 7)
# g_apsp.agregar_arista(1, 0, 8)
# g_apsp.agregar_arista(1, 2, 2)
# g_apsp.agregar_arista(2, 0, 5)
# g_apsp.agregar_arista(2, 3, 1)
# g_apsp.agregar_arista(3, 0, 2)

# dist_matrix, neg_cycle = floyd_warshall(g_apsp)
# if neg_cycle:
#     print("Ciclo negativo detectado!")
# else:
#     print("Matriz de distancias más cortas:")
#     print(dist_matrix)
#     # Expected:

```

```
# # [[0., 3., 5., 6.],
# # [8., 0., 2., 3.],
# # [5., 8., 0., 1.],
# # [2., 5., 7., 0.]]
```

3.4.4 Análisis de Complejidad

- **Tiempo:** $O(V^3)$, debido a los tres bucles anidados sobre el número de vértices.
- **Espacio:** $O(V^2)$ para almacenar la matriz de distancias.

3.4.5 Ventajas y Desventajas

- **Ventaja:** Resuelve APSP, puede manejar pesos negativos (sin ciclos negativos) y detecta ciclos negativos. Es conceptualmente simple de entender e implementar.
- **Desventaja:** Alta complejidad temporal ($O(V^3)$), lo que lo hace impráctico para grafos grandes. No reconstruye los caminos directamente sin una matriz auxiliar de predecesores.

3.5 Algoritmo A* (A-Star)

El algoritmo A* es una extensión de Dijkstra, utilizado para encontrar el camino más corto entre dos vértices específicos en un grafo. Se destaca por su eficiencia en la práctica al utilizar una **función heurística** que guía la búsqueda hacia el destino.

3.5.1 Concepto y Funcionamiento

A* combina la “distancia real” recorrida desde el inicio ($g(n)$) con una “distancia estimada” al destino ($h(n)$ - la heurística). La función de evaluación es $f(n) = g(n) + h(n)$. Prioriza la exploración de nodos que parecen estar en un camino prometedor hacia el objetivo.

- **$g(n)$:** Costo del camino desde el nodo inicial hasta el nodo actual n .
- **$h(n)$:** Estimación del costo del camino más barato desde el nodo actual n hasta el nodo objetivo. Esta es la función heurística. Para que A* garantice el camino más corto, $h(n)$ debe ser **admisible** (nunca sobreestima el costo real) y preferiblemente **monótona** (consistente).
- **$f(n)$:** Costo total estimado del camino más barato a través de n hasta el objetivo.

3.5.2 Algoritmo Detallado

Similar a Dijkstra, pero la cola de prioridad ordena por $f(n)$.

1. `open_set`: Cola de prioridad (min-heap) que contiene los nodos a evaluar, ordenados por $f(n)$.
2. `g_score[v] = infinity` para todos los v , `g_score[inicio] = 0`.

3. $f_score[v] = \text{infinity}$ para todos los v , $f_score[\text{inicio}] = h(\text{inicio}, \text{destino})$.
4. $\text{came_from}[v] = \text{undefined}$ para todos los v .
5. $\text{open_set.push}((f_score[\text{inicio}], \text{inicio}))$.
6. Mientras open_set no esté vacía:
 - a. $\text{current_f}, \text{current} = \text{open_set.pop_min}()$.
 - b. Si $\text{current} == \text{destino}$, reconstruir camino y retornar.
 - c. Para cada vecino neighbor de current con peso cost_to_neighbor :
 - i. $\text{tentative_g_score} = g_score[\text{current}] + \text{cost_to_neighbor}$.
 - ii. Si $\text{tentative_g_score} < g_score[\text{neighbor}]$:
 - $\text{came_from}[\text{neighbor}] = \text{current}$.
 - $g_score[\text{neighbor}] = \text{tentative_g_score}$.
 - $f_score[\text{neighbor}] = g_score[\text{neighbor}] + h(\text{neighbor}, \text{destino})$.
 - $\text{open_set.push}((f_score[\text{neighbor}], \text{neighbor}))$.
7. Si open_set se vacía y no se llegó al destino, no hay camino.

3.5.3 Implementación en Python

Requiere una función heurística $h(\text{actual}, \text{destino})$. Para un mapa, podría ser la distancia euclidiana.

```
import heapq
from typing import List, Tuple, Dict, Optional, Callable

# Asumimos GrafoListaAdyacencia y la función reconstruir_camino del Cap
# 1/3

def a_star(grafo, inicio: int, destino: int,
            h_func: Callable[[int, int], float]) \
    -> Tuple[List[float], List[Optional[int]]]:
    """
    Implementación del algoritmo A* para encontrar el camino más corto
    entre un inicio y un destino usando una heurística.
    :param grafo: Una instancia de GrafoListaAdyacencia.
    :param inicio: Vértice de inicio.
    :param destino: Vértice de destino.
    :param h_func: Función heurística  $h(u, v)$  que estima la distancia de
    u a v.
    Debe ser admisible y consistente para la optimalidad.
    :return: Una tupla (distancias_g, predecesores).
    distancias_g[v] es el costo del camino más corto desde
    inicio a v.
    predecesores[v] es el vértice anterior a v.
    """
    n = grafo.n
    g_score = [float('inf')] * n # Costo real desde inicio a n
    g_score[inicio] = 0

    # f_score[n] = g_score[n] + h_func(n, destino)
    f_score = [float('inf')] * n
    f_score[inicio] = h_func(inicio, destino)
```

```

predecesores: List[Optional[int]] = [None] * n

# open_set: Cola de prioridad (f_score, vertice)
open_set: List[Tuple[float, int]] = [(f_score[inicio], inicio)]

while open_set:
    current_f, current = heapq.heappop(open_set)

    if current == destino:
        return g_score, predecesores

    # Si ya hemos encontrado un camino mejor a 'current'
    # (por una entrada anterior en la cola de prioridad)
    if current_f > f_score[current]:
        continue

    for vecino, peso_uv in grafo.obtener_vecinos(current):
        tentative_g_score = g_score[current] + peso_uv

        if tentative_g_score < g_score[vecino]:
            predecesores[vecino] = current
            g_score[vecino] = tentative_g_score
            f_score[vecino] = g_score[vecino] + h_func(vecino,
↪ destino)
            heapq.heappush(open_set, (f_score[vecino], vecino))

    return g_score, predecesores # No se encontró camino a destino

# Ejemplo de uso (heurística simple, e.g., distancia euclidiana para un
↪ grid)
# Supongamos que los vértices tienen coordenadas (x,y)
# class GrafoConCoords(GrafoListaAdyacencia):
#     def __init__(self, n_v: int, coords: List[Tuple[float, float]],
↪ dirigido=False):
#         super().__init__(n_v, dirigido)
#         self.coords = coords

# def distancia_euclidiana(u: int, v: int, grafo_con_coords:
↪ GrafoConCoords) -> float:
#     x1, y1 = grafo_con_coords.coords[u]
#     x2, y2 = grafo_con_coords.coords[v]
#     return ((x1 - x2)**2 + (y1 - y2)**2)**0.5

# # Ejemplo de un grafo con 4 nodos y coordenadas
# coords_ej = [(0,0), (1,0), (1,1), (0,1)]
# g_a_star = GrafoConCoords(4, coords_ej, dirigido=False)
# g_a_star.agregar_arista(0, 1, 1)
# g_a_star.agregar_arista(1, 2, 1)
# g_a_star.agregar_arista(2, 3, 1)

```



```
# g_a_star.agregar_arista(3, 0, 1) # Es un cuadrado

# Definir la heurística para este ejemplo
# h_ej = lambda u, v: distancia_euclidiana(u, v, g_a_star)

# dists_g, prevs_a_star = a_star(g_a_star, 0, 2, h_ej)
# print(f"Distancia más corta de 0 a 2 (A*): {dists_g[2]}") # Debería ser
# ↪ 2
# print(f"Camino de 0 a 2 (A*): {reconstruir_camino(prevs_a_star, 0,
# ↪ 2)}") # [0, 1, 2] o [0, 3, 2]
```

3.5.4 Análisis de Complejidad

- **Tiempo:** La complejidad de A* es sensible a la calidad de la heurística. En el peor caso (heurística no informativa), es similar a Dijkstra: $O(E \log V)$. Con una buena heurística, puede ser significativamente más rápido, acercándose a $O(V)$ en casos ideales.
- **Espacio:** $O(V + E)$ para almacenar g_score, f_score, predecesores y la cola de prioridad.

3.5.5 Propiedades y Usos

- **Optimalidad:** A* es óptimo (encuentra el camino más corto) si la función heurística $h(n)$ es **admisibile** (nunca sobreestima el costo real) y el costo de las aristas es no negativo. Si $h(n)$ también es **consistente** (monótona), A* es aún más eficiente.
 - **Complejitud:** Si existe un camino, A* lo encontrará.
 - **Aplicaciones:** Navegación en mapas (Google Maps, Waze), planificación de rutas en videojuegos, robótica y pathfinding en IA.
-

Capítulo 4

Árboles de Expansión Mínima (MST)

Este capítulo introduce el concepto fundamental de los Árboles de Expansión Mínima (MST) y explora dos de los algoritmos más conocidos para encontrarlos: el algoritmo de Prim y el algoritmo de Kruskal. Entenderemos sus principios, implementaciones en Python y cuándo aplicar cada uno.

4.1 Introducción a los Árboles de Expansión Mínima

4.1.1 ¿Qué es un Árbol de Expansión?

Un **árbol de expansión (spanning tree)** de un grafo conexo no dirigido es un subgrafo que es un árbol (acíclico y conexo) y que incluye a todos los vértices del grafo original. Si el grafo no es conexo, no tiene un único árbol de expansión, sino un *bosque de expansión* (spanning forest).

- **Propiedades de un Árbol de Expansión:**
 - Contiene todos los V vértices del grafo original.
 - Es conexo.
 - Es acíclico.
 - Contiene exactamente $V - 1$ aristas.

4.1.2 El Problema del Árbol de Expansión Mínima (MST)

En un grafo conexo, no dirigido y ponderado, un **Árbol de Expansión Mínima (MST - Minimum Spanning Tree)** es un árbol de expansión cuyas aristas suman el menor peso posible.

- **Aplicaciones:** Los MST son fundamentales en problemas de diseño de redes, como:
 - Diseño de redes de comunicación, eléctricas o de tuberías para minimizar costos.
 - Clustering de datos (agrupamiento).
 - Circuitos impresos.

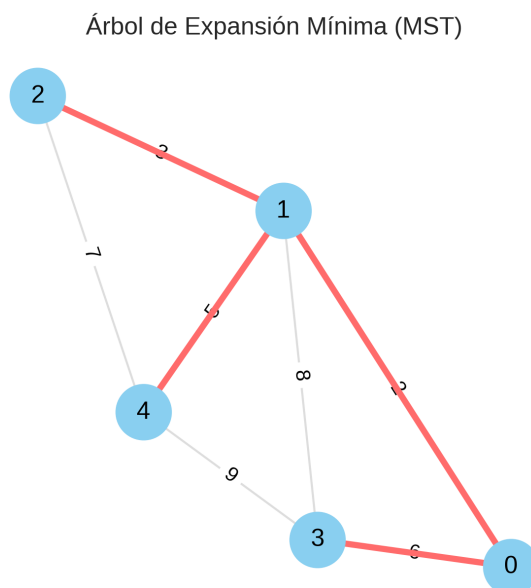


Figura 4.1: Ejemplo de MST (aristas rojas)

4.1.3 Propiedades Clave del MST

Dos propiedades son cruciales para el funcionamiento de los algoritmos de MST:

1. **Propiedad del Corte (Cut Property):** Para cualquier corte (partición de los vértices del grafo en dos conjuntos no vacíos), si una arista tiene un peso estrictamente menor que cualquier otra arista que cruza el corte, entonces esa arista debe pertenecer a *todo* MST del grafo. Si hay varias aristas con el mismo peso mínimo, al menos una de ellas pertenece a un MST.
2. **Propiedad del Ciclo (Cycle Property):** Si una arista tiene un peso estrictamente mayor que cualquier otra arista en un ciclo, entonces esa arista no puede pertenecer a *ningún* MST del grafo. Si hay varias aristas con el mismo peso máximo, al menos una de ellas no pertenece a ningún MST.

4.2 Algoritmo de Prim

El algoritmo de Prim es un algoritmo “codicioso” que construye un MST de forma incremental. Comienza desde un vértice arbitrario y va añadiendo la arista de menor peso que conecta el árbol de expansión parcial con un vértice que aún no está en él.

4.2.1 Concepto y Funcionamiento

Prim se enfoca en hacer crecer un solo componente conexo (el MST parcial) hasta que abarque todos los vértices.

1. Inicialización:

- Elija un vértice de inicio arbitrario y añádalo al MST.
- Mantenga un registro de la arista de menor peso que conecta cada vértice fuera del MST al MST.

- Utilice una **cola de prioridad** (min-heap) para almacenar las aristas candidatas que conectan un vértice fuera del MST al MST parcial.

2. Iteración:

- Extraiga de la cola de prioridad la arista de menor peso (u, v) donde u está en el MST parcial y v no.
- Añada v y la arista (u, v) al MST.
- Para cada vecino x de v que aún no está en el MST, actualice su arista candidata si se ha encontrado un camino más corto a través de v .

4.2.2 Algoritmo Detallado

1. $\text{min_cost}[v] = \text{infinity}$ para todos los v , excepto $\text{min_cost}[s] = 0$ (s es el vértice de inicio).
2. $\text{parent}[v] = \text{None}$ para todos los v .
3. $\text{in_mst}[v] = \text{False}$ para todos los v .
4. $\text{min_heap} = [(0, s)]$ (costo, vértice).
5. $\text{mst_edges} = []$
6. Mientras min_heap no esté vacía Y $\text{len}(\text{mst_edges}) < V-1$:
 - a. $\text{cost_u}, u = \text{heapq.heappop}(\text{min_heap})$.
 - b. Si $\text{in_mst}[u]$, continuar (ya procesado).
 - c. $\text{in_mst}[u] = \text{True}$.
 - d. Si $\text{parent}[u]$ no es None , añadir ($\text{parent}[u], u, \text{cost_u}$) a mst_edges .
 - e. Para cada arista (u, v) con peso w :
 - i. Si not $\text{in_mst}[v]$ Y $w < \text{min_cost}[v]$ (o $\text{cost_u} + w < \text{min_cost}[v]$ si se usa variante de Dijkstra):
 - $\text{min_cost}[v] = w$ (o $\text{cost_u} + w$).
 - $\text{parent}[v] = u$.
 - $\text{heapq.heappush}(\text{min_heap}, (\text{min_cost}[v], v))$.

4.2.3 Implementación en Python

Se utiliza `heapq` para la cola de prioridad.

```
import heapq
from typing import List, Tuple, Optional, NamedTuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1
# y sus vecinos retornan (id_vecino, peso)

class MSTEdge(NamedTuple):
    u: int
    v: int
    peso: float

def prim(grafo, inicio: int = 0) -> Tuple[List[MSTEdge], float]:
    """
    Calcula el Árbol de Expansión Mínima (MST) usando el algoritmo de
    ↪ Prim.
    :param grafo: Una instancia de GrafoListaAdyacencia.
                  Debe ser no dirigido.
    :param inicio: El vértice de inicio (entero).
    :return: Una tupla (mst_aristas, costo_total_mst).
    """
```

```

        mst_aristas es una lista de objetos MSTEdge.
        costo_total_mst es la suma de los pesos de las aristas del
↪ MST.
    """
    n = grafo.n
    min_costo_a_mst = [float('inf')] * n
    vertice_padre = [None] * n
    en_mst = [False] * n

    # Cola de prioridad: (costo_arista_al_mst, vertice)
    # Inicialmente, solo el vértice de inicio tiene costo 0 para entrar
    ↪ al MST.
    pq: List[Tuple[float, int]] = [(0, inicio)]
    min_costo_a_mst[inicio] = 0

    mst_aristas: List[MSTEdge] = []
    costo_total_mst = 0.0

    while pq and len(mst_aristas) < n - 1:
        costo_u, u = heapq.heappop(pq)

        if en_mst[u]:
            continue

        en_mst[u] = True
        costo_total_mst += costo_u
        if vertice_padre[u] is not None:
            # Añadir la arista que conectó `u` al MST
            mst_aristas.append(
                MSTEdge(vertice_padre[u], u, costo_u))

        # Relajar aristas de `u` a sus vecinos
        for v, peso_uv in grafo.obtener_vecinos(u):
            if not en_mst[v] and peso_uv < min_costo_a_mst[v]:
                min_costo_a_mst[v] = peso_uv
                vertice_padre[v] = u
                heapq.heappush(pq, (peso_uv, v))

    # Verificar si el grafo es conexo (si se encontró un MST completo)
    if len(mst_aristas) == n - 1:
        return mst_aristas, costo_total_mst
    else:
        # No se pudo formar un MST que cubra todos los vértices
        # (grafo no conexo o error)
        return [], float('inf')

# Ejemplo de uso (asumiendo GrafoListaAdyacencia del Cap 1, no dirigido)
# g_prim = GrafoListaAdyacencia(5, dirigido=False)
# g_prim.agregar_arista(0, 1, 2)

```

```
# g_prim.agregar_arista(0, 3, 6)
# g_prim.agregar_arista(1, 2, 3)
# g_prim.agregar_arista(1, 3, 8)
# g_prim.agregar_arista(1, 4, 5)
# g_prim.agregar_arista(2, 4, 7)
# g_prim.agregar_arista(3, 4, 9)

# mst_aristas, costo = prim(g_prim, 0)
# print(f"Aristas del MST de Prim: {mst_aristas}")
# # Expected: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]
# print(f"Costo total del MST: {costo}") # Expected: 16.0
```

4.2.4 Análisis de Complejidad

- **Tiempo:** $O(E \log V)$ si se usa una cola de prioridad basada en min-heap binario (como `heapq` de Python). Cada vez que se añade un vértice al MST, se pueden añadir hasta $\text{grado}(u)$ aristas a la cola de prioridad. Las operaciones de heap cuestan $O(\log E)$ o $O(\log V)$ si las aristas duplicadas se gestionan con cuidado.
- **Espacio:** $O(V + E)$ para la cola de prioridad, las listas de costos y padres.

4.2.5 Limitaciones

- Requiere un grafo conexo. Si el grafo no es conexo, Prim encontrará un MST para la componente conexa del vértice de inicio.
- Funciona solo para grafos no dirigidos.

4.3 Algoritmo de Kruskal

El algoritmo de Kruskal es otro algoritmo “codicioso” para encontrar el MST. A diferencia de Prim, que expande un solo componente, Kruskal construye el MST añadiendo las aristas de menor peso en todo el grafo, siempre y cuando no formen un ciclo con las aristas ya elegidas.

4.3.1 Concepto y Funcionamiento

Kruskal se basa en la propiedad del ciclo: una arista solo se añade si no forma un ciclo. Para verificar rápidamente si añadir una arista formaría un ciclo, se utiliza una estructura de datos llamada **Union-Find (Disjoint Set Union - DSU)**.

1. Inicialización:

- Coloque cada vértice en su propio conjunto disjunto.
- Ordene todas las aristas del grafo en orden ascendente de peso.
- Inicialice una lista vacía para las aristas del MST.

2. Iteración:

- Para cada arista (u, v) (con peso w) en el orden ascendente:
 - Si u y v pertenecen a conjuntos disjuntos diferentes (es decir, añadir la arista no forma un ciclo):

- * Añada la arista (u, v) al MST.
- * Una los conjuntos que contienen a u y v (operación union).
- Deténgase cuando el MST tenga $V - 1$ aristas o cuando se hayan considerado todas las aristas.

4.3.2 La Estructura de Datos Union-Find (DSU)

Union-Find es una estructura de datos que gestiona un conjunto de elementos particionados en un número de conjuntos disjuntos. Ofrece dos operaciones clave:

- **find(i):** Determina a qué conjunto pertenece el elemento i (normalmente retorna el representante del conjunto).
- **union(i, j):** Une los dos conjuntos que contienen a i y j en un solo conjunto.

Para optimizar DSU, se utilizan dos técnicas:

1. **Compresión de Caminos (Path Compression):** Durante la operación find, hace que cada nodo en el camino al representante apunte directamente al representante.
2. **Unión por Rango/Tamaño (Union by Rank/Size):** Al unir dos conjuntos, adjunta el árbol más pequeño bajo la raíz del árbol más grande, manteniendo los árboles planos y reduciendo la altura.

4.3.2.1 Implementación de Union-Find en Python

```
class UnionFind:
    """
    Implementación de la estructura de datos Union-Find (Disjoint Set
    Union - DSU)
    con compresión de caminos y unión por tamaño/rango.
    """
    def __init__(self, n_elementos: int):
        self.padre = list(range(n_elementos))
        self.tamano = [1] * n_elementos # Usamos tamaño para la unión por
        tamaño

    def find(self, i: int) -> int:
        """
        Encuentra el representante (raíz) del conjunto al que pertenece i.
        Realiza compresión de caminos.
        """
        if self.padre[i] == i:
            return i
        self.padre[i] = self.find(self.padre[i])
        return self.padre[i]

    def union(self, i: int, j: int) -> bool:
        """
        Une los conjuntos que contienen a i y j.
        Retorna True si los conjuntos eran diferentes y se unieron, False
        si ya
        estaban en el mismo conjunto.
        """
```



```

Realiza unión por tamaño.
"""
root_i = self.find(i)
root_j = self.find(j)

if root_i != root_j:
    # Unir el árbol más pequeño bajo la raíz del más grande
    if self.tamano[root_i] < self.tamano[root_j]:
        self.padre[root_i] = root_j
        self.tamano[root_j] += self.tamano[root_i]
    else:
        self.padre[root_j] = root_i
        self.tamano[root_i] += self.tamano[root_j]
    return True
return False

# Ejemplo de uso UnionFind
# uf = UnionFind(5)
# uf.union(0, 1) # {0,1}, {2}, {3}, {4}
# uf.union(2, 3) # {0,1}, {2,3}, {4}
# uf.union(0, 2) # {0,1,2,3}, {4}
# print(uf.find(1) == uf.find(3)) # True
# print(uf.find(1) == uf.find(4)) # False

```

4.3.3 Implementación de Kruskal en Python

```

from typing import List, Tuple, NamedTuple

# Asumimos que GrafoListaAdyacencia está definido como en el Capítulo 1
# y que la clase UnionFind está definida arriba.

# Usaremos la misma NamedTuple MSTEdge para las aristas del MST

def kruskal(grafo) -> Tuple[List[MSTEdge], float]:
    """
    Calcula el Árbol de Expansión Mínima (MST) usando el algoritmo de
    ↪ Kruskal.
    :param grafo: Una instancia de GrafoListaAdyacencia.
                    Debe ser no dirigido (el algoritmo solo añade aristas
    ↪ una vez).
    :return: Una tupla (mst_aristas, costo_total_mst).
             mst_aristas es una lista de objetos MSTEdge.
             costo_total_mst es la suma de los pesos de las aristas del
    ↪ MST.
    """
    n = grafo.n
    todas_las_aristas: List[MSTEdge] = []

    # Recopilar todas las aristas del grafo.
    # Para grafos no dirigidos, NetworkX retorna (u,v) y (v,u) si es
    ↪ bidireccional,

```

```

# pero aquí nuestra GrafoListaAdyacencia solo guarda (u,v).
# Debemos asegurarnos de no añadir aristas duplicadas (u,v) y (v,u)
# en la lista de todas_las_aristas para Kruskal.
# Una forma es usar un set para evitar duplicados o solo iterar en
# ↪ una dirección.
# Aquí asumimos que grafo.obtener_vecinos(u) para  $u < v$  es suficiente
# ↪ si
# el grafo es conceptualmente no dirigido.
aristas_procesadas = set() # Para evitar añadir (u,v) y (v,u)
for u in range(n):
    for v, peso in grafo.obtener_vecinos(u):
        if u < v: # Añadir solo una vez por arista no dirigida
            todas_las_aristas.append(MSTEdge(u, v, peso))

# 1. Ordenar todas las aristas por peso ascendente
todas_las_aristas.sort(key=lambda edge: edge.peso)

# 2. Inicializar la estructura Union-Find
uf = UnionFind(n)

mst_aristas: List[MSTEdge] = []
costo_total_mst = 0.0

# 3. Iterar sobre las aristas ordenadas
for edge in todas_las_aristas:
    u, v, peso = edge
    if uf.union(u, v): # Si u y v no están en el mismo componente y
# ↪ se unen
        mst_aristas.append(edge)
        costo_total_mst += peso
        if len(mst_aristas) == n - 1:
            break # MST completo

# Verificar si el grafo es conexo (si se encontró un MST completo)
if len(mst_aristas) == n - 1:
    return mst_aristas, costo_total_mst
else:
    # No se pudo formar un MST que cubra todos los vértices
    # (grafo no conexo o error)
    return [], float('inf')

# Ejemplo de uso (asumiendo GrafoListaAdyacencia del Cap 1, no dirigido)
# g_kruskal = GrafoListaAdyacencia(5, dirigido=False)
# g_kruskal.agregar_arista(0, 1, 2)
# g_kruskal.agregar_arista(0, 3, 6)
# g_kruskal.agregar_arista(1, 2, 3)
# g_kruskal.agregar_arista(1, 3, 8)
# g_kruskal.agregar_arista(1, 4, 5)
# g_kruskal.agregar_arista(2, 4, 7)
# g_kruskal.agregar_arista(3, 4, 9)

```

```
# mst_aristas, costo = kruskal(g_kruskal)
# print(f"Aristas del MST de Kruskal: {mst_aristas}")
# # Expected: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)] (orden puede
# ↪ variar ligeramente)
# print(f"Costo total del MST: {costo}") # Expected: 16.0
```

4.3.4 Análisis de Complejidad

- **Tiempo:** $O(E \log E)$ o $O(E \log V)$.
 - $O(E \log E)$ para ordenar las aristas.
 - $O(E \alpha(V))$ para las operaciones de Union-Find, donde α es la función inversa de Ackermann, extremadamente lenta pero en la práctica casi constante.
 - Dominado por la ordenación de las aristas. Dado que $E \leq V^2$, $E \log E$ es a lo sumo $V^2 \log V$, pero típicamente $E \log E$ es más cercano a $E \log V$.
- **Espacio:** $O(V + E)$ para almacenar la estructura Union-Find y las aristas.

4.3.5 Comparativa Prim vs. Kruskal

Característica	Algoritmo de Prim	Algoritmo de Kruskal
Enfoque	Crece un componente único	Une componentes pequeños
Estructura Clave	Cola de prioridad (Min-Heap)	Union-Find (DSU)
Grafo Ideal	Densos ($E \approx V^2$)	Dispersos ($E \approx V$)
Complejidad Temporal	$O(E \log V)$	$O(E \log E)$
Requiere Conexión	Sí (para un MST completo)	Sí (para un MST completo)
Fácil Detección de Ciclos	Implícito (crece un árbol)	Explícito (con Union-Find)

Capítulo 5

Grafos Dirigidos Acíclicos (DAGs) y Ordenamiento Topológico

Los Grafos Dirigidos Acíclicos, conocidos comúnmente como DAGs (por sus siglas en inglés, *Directed Acyclic Graphs*), ocupan un lugar especial en la teoría de grafos y la informática. Su estructura única, que representa dependencias direccionales sin ciclos, los convierte en la herramienta ideal para modelar flujos de trabajo, sistemas de compilación, análisis de datos y mucho más.

5.1 Introducción a los DAGs

5.1.1 Definición y Propiedades

Un **DAG** es un grafo dirigido que no contiene ningún ciclo dirigido. Es decir, no existe ninguna secuencia de vértices $v_1, v_2, \dots, v_k$ tal que existan aristas $(v_1, v_2), (v_2, v_3), \dots, (v_{k-1}, v_k)$ y (v_k, v_1) .

- **Orden Parcial:** Un DAG define un ordenamiento parcial sobre sus vértices. Si existe una ruta de u a v , decimos que u precede a v ($u \preceq v$). Si no hay ruta entre ellos, no son comparables.
 - **Fuentes y Sumideros:**
 - **Fuente (Source):** Un vértice con grado de entrada 0 (nadie apunta a él). Todo DAG finito tiene al menos una fuente.
 - **Sumidero (Sink):** Un vértice con grado de salida 0 (no apunta a nadie). Todo DAG finito tiene al menos un sumidero.
-

5.2 Ordenamiento Topológico

El ordenamiento topológico es la operación fundamental sobre los DAGs. Consiste en una ordenación lineal de todos los vértices del grafo tal que, para cada arista dirigida (u, v) , el vértice u aparece antes que el vértice v en la secuencia.

- **Existencia:** Un grafo tiene un ordenamiento topológico si y solo si es un DAG.
- **Unicidad:** Un DAG puede tener múltiples ordenamientos topológicos válidos. Es único si y solo si tiene un camino hamiltoniano (un camino que visita todos los vértices).

5.2.1 Algoritmo de Kahn (Basado en Grados de Entrada)

Este algoritmo es intuitivo y se basa en la idea de procesar iterativamente los nodos que no tienen dependencias pendientes.

5.2.1.1 Algoritmo Detallado

1. Calcula el **grado de entrada** (in-degree) para cada vértice.
2. Inicializa una cola con todos los vértices que tienen **grado de entrada 0** (fuentes).
3. Mientras la cola no esté vacía:
 - a. Desencola un vértice u .
 - b. Añade u a la lista de orden_topologico.
 - c. Para cada vecino v de u :
 - i. Decrementa el grado de entrada de v en 1 (simulando que eliminamos la arista $u \rightarrow v$).
 - ii. Si el grado de entrada de v llega a 0, encola v .
4. Si la longitud del orden_topologico es menor que el número de vértices, el grafo tiene un ciclo.

5.2.1.2 Implementación en Python

```
from collections import deque
from typing import List, Dict

# Asumimos GrafoListaAdyacencia del Cap 1

def ordenamiento_topologico_kahn(grafo) -> List[int]:
    """
    Realiza un ordenamiento topológico usando el algoritmo de Kahn.
    :param grafo: Instancia de GrafoListaAdyacencia (debe ser dirigido).
    :return: Lista de vértices en orden topológico.
    :raises ValueError: Si el grafo contiene un ciclo.
    """
    n = grafo.n
    grado_entrada = [0] * n

    # 1. Calcular grados de entrada
    for u in range(n):
        for v, _ in grafo.obtener_vecinos(u):
            grado_entrada[v] += 1

    # 2. Inicializar cola con fuentes
    cola = deque([u for u in range(n) if grado_entrada[u] == 0])
    orden = []
```

```

# 3. Procesar vértices
while cola:
    u = cola.popleft()
    orden.append(u)

    for v, _ in grafo.obtener_vecinos(u):
        grado_entrada[v] -= 1
        if grado_entrada[v] == 0:
            cola.append(v)

# 4. Verificar ciclos
if len(orden) != n:
    raise ValueError("El grafo contiene un ciclo, no es un DAG.")

return orden

```

5.2.2 Algoritmo Basado en DFS

Este enfoque utiliza la Búsqueda en Profundidad. La idea clave es que un vértice solo se añade a la lista ordenada cuando todos sus descendientes ya han sido visitados completamente.

5.2.2.1 Algoritmo Detallado

1. Realiza un DFS completo sobre el grafo.
2. Mantén un registro del tiempo de finalización o simplemente añade el vértice a una lista cuando la llamada recursiva de DFS termine para ese vértice.
3. El ordenamiento topológico es la lista de vértices en **orden inverso** a su tiempo de finalización.

5.2.2.2 Implementación en Python

```

def ordenamiento_topologico_dfs(grafo) -> List[int]:
    """
    Realiza un ordenamiento topológico usando DFS.
    :param grafo: Instancia de GrafoListaAdyacencia (debe ser dirigido).
    :return: Lista de vértices en orden topológico.
    :raises ValueError: Si se detecta un ciclo (back-edge).
    """
    n = grafo.n
    visitados = [0] * n # 0: no visitado, 1: visitando, 2: visitado
    orden = []

    def dfs(u):
        visitados[u] = 1 # Marcamos como visitando (gris)

        for v, _ in grafo.obtener_vecinos(u):
            if visitados[v] == 1:
                # Encontramos una arista hacia un nodo en proceso -> Ciclo
                raise ValueError("Ciclo detectado")

            if visitados[v] == 0:
                dfs(v)

        orden.append(u)
        visitados[u] = 2

    dfs(grafo.obtener_inicio())

    return orden[::-1]

```

```

        if visitados[v] == 0:
            dfs(v)

    visitados[u] = 2 # Marcamos como visitado (negro)
    # Añadimos a la lista al terminar de procesar todos sus hijos
    orden.append(u)

for i in range(n):
    if visitados[i] == 0:
        try:
            dfs(i)
        except ValueError as e:
            raise e

# El resultado es el reverso del orden de finalización
return orden[::-1]

```

5.3 Aplicaciones Prácticas

5.3.1 Resolución de Dependencias

Es el caso de uso clásico. * **Gestores de Paquetes (pip, npm):** Determinan el orden en que deben instalarse las librerías. Si A depende de B, B debe instalarse antes. * **Sistemas de Construcción (Make, Gradle):** Compilan primero los archivos fuente base antes que los que dependen de ellos.

5.3.2 Planificación de Tareas (Scheduling)

Si tienes un conjunto de tareas donde algunas deben completarse antes de que otras puedan comenzar, el ordenamiento topológico te da una secuencia válida de ejecución. Esto se usa en diagramas de PERT y gestión de proyectos.

5.4 Caminos Más Largos y Ruta Crítica

En grafos generales, encontrar el camino simple más largo es un problema **NP-difícil** (equivalente al problema del Camino Hamiltoniano). Sin embargo, en **DAGs**, este problema se puede resolver eficientemente en tiempo lineal $O(V + E)$.

Esto es crucial para el **Método de la Ruta Crítica (CPM)** en gestión de proyectos: la duración mínima de un proyecto está determinada por el camino más largo de dependencias secuenciales.

5.4.1 Algoritmo usando Ordenamiento Topológico

La propiedad clave es que, si procesamos los vértices en orden topológico, al llegar a un vértice u , ya hemos procesado todos los posibles predecesores que podrían conducir a u .

1. Obtener el ordenamiento topológico del grafo.
2. Inicializar $\text{dist}[v] = -\text{infinito}$ para todo v , excepto $\text{dist}[\text{inicio}] = 0$.
3. Iterar por cada vértice u en orden topológico:
 - Si $\text{dist}[u]$ es finito:
 - Para cada vecino v con peso w :
 - * $\text{dist}[v] = \max(\text{dist}[v], \text{dist}[u] + w)$ (Relajación para máximo).

5.4.1.1 Implementación en Python

```
def camino_mas_largo_dag(grafo, inicio: int) -> List[float]:  
    """  
    Calcula la distancia del camino más largo desde 'inicio' a todos  
    los demás nodos en un DAG.  
    """  
    try:  
        orden_topo = ordenamiento_topologico_kahn(grafo)  
    except ValueError:  
        return [] # No es un DAG  
  
    distancias = [float('-inf')] * grafo.n  
    distancias[inicio] = 0  
  
    # Procesar nodos en orden topológico garantiza que cuando procesamos  
    # ↪ u,  
    # ya tenemos la distancia máxima correcta hacia u.  
    for u in orden_topo:  
        if distancias[u] != float('-inf'):  
            for v, peso in grafo.obtener_vecinos(u):  
                if distancias[u] + peso > distancias[v]:  
                    distancias[v] = distancias[u] + peso  
  
    return distancias
```


Capítulo 6

Conectividad y Componentes Fuertemente Conexas (SCC)

La conectividad es una propiedad fundamental que determina qué tan robusta o integrada es una red. En este capítulo, exploraremos cómo identificar las partes críticas de un grafo, como los “puntos de fallo único” (puentes y puntos de articulación), y cómo descomponer grafos dirigidos en sus estructuras cíclicas básicas: las Componentes Fuertemente Conexas (SCC).

6.1 Conectividad en Grafos No Dirigidos

En un grafo no dirigido, la conectividad se refiere a la capacidad de llegar de cualquier vértice a cualquier otro.

6.1.1 Conceptos Clave

- **Componente Conexo:** Un subconjunto maximal de vértices tal que existe un camino entre cada par de ellos.
- **Punto de Articulación (Cut Vertex):** Un vértice que, si se elimina (junto con sus aristas incidentes), aumenta el número de componentes conexas del grafo. Representan vulnerabilidades o “cuellos de botella”.
- **Puente (Bridge):** Una arista que, si se elimina, aumenta el número de componentes conexas. En una red de computadoras, un puente es un cable crítico cuya falla desconecta la red.

6.1.2 Algoritmo para Encontrar Puentes (Tarjan)

El método ingenuo para encontrar puentes consiste en eliminar cada arista una por una y verificar con BFS/DFS si el grafo sigue conexo ($O(E \cdot (V + E))$). Sin embargo, podemos hacerlo mucho más rápido, en $O(V + E)$, usando un solo recorrido DFS.

6.1.2.1 Lógica del Algoritmo (Discovery Time y Low-Link)

Para cada nodo u en el árbol DFS, mantenemos dos valores:

1. **disc[u] (Discovery Time):** El tiempo (o contador) en el que el nodo u fue visitado por primera vez durante el DFS.
2. **low[u] (Low-Link Value):** El menor disc alcanzable desde u (incluido él mismo) en el subárbol DFS de u , posiblemente utilizando una **arista de retroceso** (back-edge), pero no la arista directa hacia su padre.

Condición de Puente: Una arista (u, v) es un puente si y solo si $\text{low}[v] > \text{disc}[u]$.

Esto significa que no hay ninguna arista de retroceso desde el subárbol de v que conecte con u o con cualquiera de sus ancestros. Por lo tanto, la única forma de llegar a v desde u es a través de la arista (u, v) .

6.1.2.2 Implementación en Python

```
def encontrar_puentes(grafo) -> list[tuple[int, int]]:
    """
    Encuentra todos los puentes en un grafo no dirigido usando DFS.
    :param grafo: Instancia de GrafoListaAdyacencia (no dirigido).
    :return: Lista de tuplas (u, v) representando los puentes.
    """
    n = grafo.n
    disc = [-1] * n
    low = [-1] * n
    tiempo = 0
    puentes = []

    def dfs(u: int, padre: int = -1):
        nonlocal tiempo
        disc[u] = low[u] = tiempo
        tiempo += 1

        for v, _ in grafo.obtener_vecinos(u):
            if v == padre:
                continue

            if disc[v] != -1:
                # v ya visitado: es una arista de retroceso
                low[u] = min(low[u], disc[v])
            else:
                # v no visitado: es una arista del árbol (tree-edge)
                dfs(v, u)

                # Al regresar, actualizamos low[u] basado en el hijo
                low[u] = min(low[u], low[v])

                # Chequeo de puente
                if low[v] > disc[u]:
                    puentes.append((u, v))

    for i in range(n):
        if disc[i] == -1:
```

```

        dfs(i)

    return puentes

# Ejemplo de uso
# g = GrafoListaAdyacencia(5, dirigido=False)
# g.agregar_arista(1, 0); g.agregar_arista(0, 2)
# g.agregar_arista(2, 1); g.agregar_arista(0, 3)
# g.agregar_arista(3, 4)
# print(encontrar_puentes(g))
# Salida esperada: [(3, 4), (0, 3)]

```

6.2 Conectividad en Grafos Dirigidos

En grafos dirigidos, la conectividad es más compleja debido a la dirección de las aristas.

6.2.1 Componentes Fuertemente Conexas (SCC)

Una **Componente Fuertemente Conexa (SCC)** es un subgrafo maximal donde para cada par de vértices u, v , existe un camino de u a v **Y** un camino de v a u .

Si contraemos cada SCC en un solo super-nodo, el grafo resultante es siempre un **DAG** (Grafo Acíclico Dirigido), conocido como el **Grafo de Condensación**.

6.2.2 Algoritmo de Kosaraju

Existen varios algoritmos lineales ($O(V + E)$) para encontrar SCCs (Tarjan, Kosaraju, Gabow). El algoritmo de **Kosaraju** es conceptualmente el más sencillo de entender y se basa en dos pasadas de DFS.

6.2.2.1 Pasos del Algoritmo

1. **Primera Pasada (DFS):** Realizar un DFS completo sobre el grafo original G y almacenar los vértices en una pila en orden de **finalización** (cuando la llamada recursiva retorna). El nodo que termina último estará en el tope.
2. **Transponer el Grafo (G^T):** Crear un nuevo grafo con las mismas aristas pero en dirección opuesta (si $u \rightarrow v$ existe en G , entonces $v \rightarrow u$ existe en G^T).
3. **Segunda Pasada (DFS en G^T):** Procesar los vértices uno a uno sacándolos de la pila creada en el paso 1. Si un vértice no ha sido visitado en esta segunda pasada, iniciar un DFS desde él en G^T . Todos los vértices alcanzables en este DFS forman una nueva SCC.

6.2.2.2 Implementación en Python

```

def encontrar_scc_kosaraju(grafo) -> list[list[int]]:
    """

```

```

Encuentra las Componentes Fuertemente Conexas (SCC) usando el
algoritmo de Kosaraju.
:param grafo: Instancia de GrafoListaAdyacencia (dirigido).
:return: Lista de listas, donde cada sublista es una SCC.
"""
n = grafo.n
visitados = [False] * n
pila = []

# 1. Primera pasada: Llenar la pila por orden de finalización
def dfs_llenar_pila(u):
    visitados[u] = True
    for v, _ in grafo.obtener_vecinos(u):
        if not visitados[v]:
            dfs_llenar_pila(v)
    pila.append(u)

for i in range(n):
    if not visitados[i]:
        dfs_llenar_pila(i)

# 2. Transponer el grafo (invertir aristas)
# Nota: Esto requiere acceso a la estructura interna o un método
# `transponer`. Aquí lo construimos manualmente para el ejemplo.
grafo_t = {i: [] for i in range(n)}
for u in range(n):
    for v, _ in grafo.obtener_vecinos(u):
        grafo_t[v].append(u) # Arista v -> u

# 3. Segunda pasada: DFS en el grafo transpuesto
visitados = [False] * n
sccs = []

def dfs_scc(u, componente_actual):
    visitados[u] = True
    componente_actual.append(u)
    for v in grafo_t[u]:
        if not visitados[v]:
            dfs_scc(v, componente_actual)

while pila:
    u = pila.pop()
    if not visitados[u]:
        componente_actual = []
        dfs_scc(u, componente_actual)
        sccs.append(componente_actual)

return sccs

# Ejemplo de uso

```

```
# g_dir = GrafoListaAdyacencia(5, dirigido=True)
# g_dir.agregar_arista(1, 0)
# g_dir.agregar_arista(0, 2)
# g_dir.agregar_arista(2, 1) # Ciclo 0-1-2
# g_dir.agregar_arista(0, 3)
# g_dir.agregar_arista(3, 4)
# print(encontrar_scc_kosaraju(g_dir))
# Salida esperada: [[0, 2, 1], [3], [4]] (el orden dentro de SCC puede
↪ variar)
```

6.3 Aplicaciones Prácticas

6.3.1 Análisis de Redes y Vulnerabilidad

La identificación de puntos de articulación y puentes es crucial en el diseño de redes de telecomunicaciones, eléctricas o de transporte. * **Robustez:** Una red es robusta si no tiene puntos de articulación (es decir, es biconexa). Esto garantiza que la falla de un solo nodo no desconectará la red. * **Diseño:** Al diseñar una red, se busca minimizar el número de puentes añadiendo redundancia estratégica.

6.3.2 Resolución de 2-Satisfacibilidad (2-SAT)

El problema de satisfacibilidad booleana (SAT) es en general NP-completo. Sin embargo, el caso especial **2-SAT** (donde cada cláusula tiene a lo sumo 2 literales) se puede resolver en tiempo polinomial usando SCCs.

1. Se construye un grafo de implicación donde las variables y sus negaciones son nodos. La cláusula $(A \vee B)$ equivale a $(\neg A \implies B)$ y $(\neg B \implies A)$.
2. Se calculan las SCCs del grafo.
3. Si una variable x y su negación $\neg x$ están en la misma SCC, la fórmula es insatisfacible (porque $x \implies \dots \implies \neg x \implies \dots \implies x$, una contradicción).
4. Si no, se puede encontrar una asignación de verdad válida usando el orden topológico de las SCCs.

6.3.3 Optimización de Consultas en Grafos Web

En el análisis de la web, las SCCs ayudan a agrupar páginas que se enlazan fuertemente entre sí. El “Grafo de la Web” se puede simplificar contrayendo millones de páginas en sus respectivas SCCs, resultando en una estructura “Bow-tie” (Moño) que facilita el análisis macroscópico y algoritmos como PageRank.

Capítulo 7

Flujo Máximo y Matching

El problema del flujo máximo es uno de los pilares de la optimización combinatoria. Se ocupa de encontrar la mayor cantidad de “flujo” (agua, tráfico, datos) que se puede enviar a través de una red con capacidades limitadas en sus enlaces. Este capítulo explora los algoritmos clásicos para resolverlo y cómo estos se aplican sorprendentemente a problemas de emparejamiento (matching).

7.1 Redes de Flujo

Una **red de flujo** es un grafo dirigido $G = (V, E)$ donde cada arista (u, v) tiene una capacidad no negativa $c(u, v) \geq 0$. Distinguimos dos vértices especiales: *

- * **Fuente (s)**: Un vértice sin aristas entrantes (idealmente), donde se origina el flujo.
- * **Sumidero (t)**: Un vértice sin aristas salientes (idealmente), donde termina el flujo.

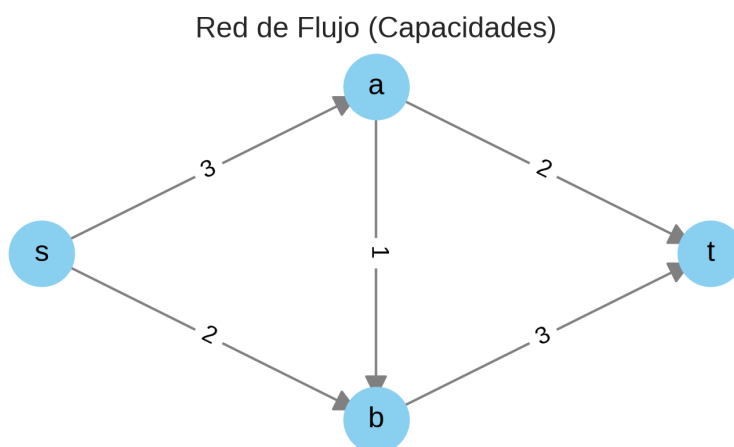


Figura 7.1: Red de Flujo con Capacidades

7.1.1 Propiedades del Flujo

Un **flujo** es una función $f(u, v)$ que asigna un valor a cada arista, cumpliendo dos condiciones:

1. **Restricción de Capacidad:** Para toda arista, el flujo no puede superar la capacidad: $0 \leq f(u, v) \leq c(u, v)$.
2. **Conservación del Flujo:** Para todo vértice u (excepto la fuente y el sumidero), el flujo total que entra debe ser igual al flujo total que sale: $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$.

El valor del flujo total de la red es la cantidad total que sale de la fuente hacia el sumidero. El objetivo es maximizar este valor.

7.2 Algoritmo de Ford-Fulkerson

El método de Ford-Fulkerson es un enfoque general para resolver el problema del flujo máximo. Se basa en la idea de encontrar caminos por donde todavía se puede enviar más flujo y saturarlos progresivamente.

7.2.1 Conceptos Clave

- **Red Residual (G_f):** Un grafo que indica cuánto flujo adicional se puede enviar. Si por una arista (u, v) pasa un flujo $f(u, v)$ y tiene capacidad $c(u, v)$:
 - La **capacidad residual** directa es $c_f(u, v) = c(u, v) - f(u, v)$.
 - Aparece una **arista de retroceso** (v, u) con capacidad residual $f(u, v)$, que representa la posibilidad de “cancelar” o devolver flujo.
- **Camino Aumentante:** Un camino simple desde la fuente s al sumidero t en la red residual G_f , donde todas las aristas tienen capacidad residual positiva.

7.2.2 Algoritmo Detallado

1. Inicializar el flujo en todas las aristas a 0.
 2. Mientras exista un camino aumentante p en la red residual G_f :
 - Encontrar la capacidad residual mínima b (“cuello de botella”) a lo largo del camino p .
 - Aumentar el flujo a lo largo de p en b :
 - Para cada arista (u, v) en p :
 - * Incrementar $f(u, v)$ en b .
 - * Decrementar $f(v, u)$ en b (en la lógica de la red residual).
 3. Retornar el flujo total acumulado.
-

7.3 Algoritmo de Edmonds-Karp

El algoritmo de Edmonds-Karp es una implementación específica del método de Ford-Fulkerson. La diferencia clave es cómo encuentra el camino aumentante: Edmonds-Karp utiliza **BFS (Búsqueda en Amplitud)** para encontrar siempre el camino aumentante con el **menor número de aristas**.

Esta simple elección garantiza que el algoritmo termine y tenga una complejidad temporal polinomial de $O(VE^2)$, a diferencia de Ford-Fulkerson genérico (usando

DFS), que puede depender del valor del flujo máximo y ser muy lento con capacidades grandes irracionales.

7.3.1 Implementación en Python

```
from collections import deque

class GrafoFlujo:
    """
    Grafo dirigido para problemas de flujo máximo.
    Soporta capacidades en las aristas.
    """
    def __init__(self, n_vertices: int):
        self.n = n_vertices
        # Matriz de capacidades para acceso rápido.
        # graph[u][v] almacena la capacidad residual de u a v.
        self.capacity = [[0] * n_vertices for _ in range(n_vertices)]
        # Lista de adyacencia para iterar eficientemente sobre vecinos
        self.graph = {i: [] for i in range(n_vertices)}

    def agregar_arista(self, u: int, v: int, capacidad: int):
        # Arista directa con capacidad
        self.graph[u].append(v)
        self.graph[v].append(u) # Arista inversa para el grafo residual
        self.capacity[u][v] = capacidad

    def bfs(self, s: int, t: int, parent: list) -> bool:
        """
        Busca un camino aumentante usando BFS.
        Llena el array 'parent' para reconstruir el camino.
        """
        visited = [False] * self.n
        queue = deque([s])
        visited[s] = True
        parent[s] = -1

        while queue:
            u = queue.popleft()

            for v in self.graph[u]:
                # Si no visitado y hay capacidad residual
                if not visited[v] and self.capacity[u][v] > 0:
                    queue.append(v)
                    visited[v] = True
                    parent[v] = u
                    if v == t:
                        return True

        return False

    def edmonds_karp(self, source: int, sink: int) -> int:
        """
```

```

Calcula el flujo máximo desde source a sink.
"""
parent = [-1] * self.n
max_flow = 0

# Mientras exista un camino aumentante en el grafo residual
while self.bfs(source, sink, parent):
    # Encontrar el cuello de botella (flujo mínimo) en el camino
    path_flow = float('inf')
    v = sink
    while v != source:
        u = parent[v]
        path_flow = min(path_flow, self.capacity[u][v])
        v = u

    # Actualizar capacidades residuales
    max_flow += path_flow
    v = sink
    while v != source:
        u = parent[v]
        self.capacity[u][v] -= path_flow # Reducir capacidad
        ↪ directa
        self.capacity[v][u] += path_flow # Aumentar capacidad
        ↪ inversa
        v = u

return max_flow

# Ejemplo de uso
# g_flujo = GrafoFlujo(6)
# g_flujo.agregar_arista(0, 1, 16)
# g_flujo.agregar_arista(0, 2, 13)
# g_flujo.agregar_arista(1, 2, 10)
# g_flujo.agregar_arista(1, 3, 12)
# g_flujo.agregar_arista(2, 1, 4)
# g_flujo.agregar_arista(2, 4, 14)
# g_flujo.agregar_arista(3, 2, 9)
# g_flujo.agregar_arista(3, 5, 20)
# g_flujo.agregar_arista(4, 3, 7)
# g_flujo.agregar_arista(4, 5, 4)

# print(f"Flujo Máximo: {g_flujo.edmonds_karp(0, 5)}") # Salida
↪ esperada: 23

```

7.4 Matching en Grafos Bipartitos

El problema del emparejamiento máximo (Maximum Bipartite Matching) consiste en encontrar el mayor número de aristas en un grafo bipartito tal que ningún par

de aristas comparta un vértice común.

7.4.1 Aplicaciones

- **Asignación de Tareas:** Asignar empleados a tareas, donde cada empleado puede hacer ciertas tareas.
- **Residencia Médica:** Asignar estudiantes a hospitales.

7.4.2 Reducción a Flujo Máximo

Podemos resolver este problema modelándolo como una red de flujo:

1. Crear un **super-origen** s y un **super-sumidero** t .
2. Conectar s a todos los nodos del conjunto izquierdo del grafo bipartito con aristas de capacidad 1.
3. Mantener las aristas del grafo bipartito (de izquierda a derecha) como aristas dirigidas con capacidad 1 (o infinita).
4. Conectar todos los nodos del conjunto derecho a t con aristas de capacidad 1.
5. El flujo máximo en esta red es igual al tamaño del matching máximo.

7.4.3 Teorema de Flujo Máximo - Corte Mínimo

Este teorema fundamental establece que el valor del flujo máximo de una red es igual a la capacidad del corte mínimo. Un **corte** es una partición de los vértices en dos conjuntos disjuntos, uno que contiene la fuente y otro el sumidero. La capacidad del corte es la suma de las capacidades de las aristas que van del conjunto de la fuente al del sumidero.

Esto tiene aplicaciones directas en visión por computadora (segmentación de imágenes) y minería de datos.

7.4.4 Implementación de Matching Bipartito (vía Flujo)

Usando la clase GrafoFlujo anterior:

```
def matching_bipartito(n_izq: int, n_der: int,
                      aristas: list[tuple[int, int]]) -> int:
    """
    Calcula el matching máximo en un grafo bipartito.
    :param n_izq: Número de nodos en el conjunto izquierdo.
    :param n_der: Número de nodos en el conjunto derecho.
    :param aristas: Lista de tuplas (u, v) donde u está en izq y v en der.
                    u va de 0 a n_izq-1, v va de 0 a n_der-1.
    """
    # Total nodos = fuente + n_izq + n_der + sumidero
    # IDs: fuente=0, izq=1..n_izq, der=n_izq+1..n_izq+n_der,
    #      ↪ sumidero=total-1
    total_nodes = 1 + n_izq + n_der + 1
    source = 0
    sink = total_nodes - 1

    g = GrafoFlujo(total_nodes)
```

```

# Conectar fuente a conjunto izquierdo
for i in range(n_izq):
    # Nodos izq mapeados a 1 ... n_izq
    g.agregar_arista(source, i + 1, 1)

# Conectar conjunto derecho a sumidero
for j in range(n_der):
    # Nodos der mapeados a n_izq + 1 ... n_izq + n_der
    node_der_id = n_izq + 1 + j
    g.agregar_arista(node_der_id, sink, 1)

# Conectar aristas del grafo bipartito
for u, v in aristas:
    u_id = u + 1
    v_id = n_izq + 1 + v
    g.agregar_arista(u_id, v_id, 1)

return g.edmonds_karp(source, sink)

# Ejemplo de Matching
# 3 candidatos (0,1,2), 3 trabajos (0,1,2)
# Candidato 0 puede hacer trabajo 1
# Candidato 1 puede hacer trabajo 0 y 2
# Candidato 2 puede hacer trabajo 0
# Aristas: (0,1), (1,0), (1,2), (2,0)
# matching = matching_bipartito(3, 3, [(0,1), (1,0), (1,2), (2,0)])
# print(f"Matching Máximo: {matching}") # Salida esperada: 3

```

Capítulo 8

Grafos Especiales y sus Algoritmos

No todos los grafos son iguales. Existen clases específicas de grafos que poseen estructuras y propiedades únicas. Reconocer que un problema se puede modelar con uno de estos grafos especiales es a menudo la clave para desbloquear soluciones mucho más eficientes que las disponibles para grafos generales.

En este capítulo, exploraremos tres tipos fundamentales: Árboles, Grafos Bipartitos y Grafos Planos.

8.1 Árboles

Los árboles son quizás la estructura de datos más fundamental después de los arrays y las listas enlazadas. En teoría de grafos, un árbol es un grafo no dirigido que es **conexo** y **acíclico**.

8.1.1 Propiedades Definitorias

Un grafo G con V vértices es un árbol si cumple cualquiera de las siguientes condiciones equivalentes:

- Es conexo y tiene $V - 1$ aristas.
- Es acíclico y tiene $V - 1$ aristas.
- Existe un único camino simple entre cualquier par de vértices.
- Si se añade cualquier arista, se crea exactamente un ciclo.
- Si se elimina cualquier arista, el grafo deja de ser conexo (la arista es un puente).

8.1.2 Terminología de Árboles Enraizados

Aunque los árboles en teoría de grafos no tienen dirección, en informática a menudo elegimos un nodo como **raíz** (root), lo que induce una jerarquía:

- **Raíz:** El nodo superior.
- **Padre/Hijo:** Si u está conectado a v y u está más cerca de la raíz, u es el padre de v .

- **Hoja (Leaf):** Un nodo sin hijos (grado 1, excepto si es la raíz única).
- **Altura:** La longitud del camino más largo desde la raíz hasta una hoja.
- **Profundidad:** La longitud del camino desde la raíz hasta un nodo específico.

8.1.3 Recorridos en Árboles (Tree Traversals)

A diferencia de los grafos generales donde usamos BFS/DFS genéricos, en árboles (especialmente binarios) definimos órdenes de visita específicos que son variantes de DFS.

- **Pre-orden (Pre-order):** Raíz → Izquierda → Derecha. Útil para copiar árboles.
- **In-orden (In-order):** Izquierda → Raíz → Derecha. En árboles de búsqueda binaria (BST), visita los nodos en orden ascendente.
- **Post-orden (Post-order):** Izquierda → Derecha → Raíz. Útil para eliminar árboles o evaluar expresiones matemáticas.

8.1.3.1 Implementación en Python (Árbol N-ario)

```
class TreeNode:
    def __init__(self, valor: int):
        self.valor = valor
        self.hijos: list['TreeNode'] = []

    def agregar_hijo(self, nodo_hijo: 'TreeNode'):
        self.hijos.append(nodo_hijo)

def recorrido_preorden(raiz: TreeNode, resultado: list[int]):
    if raiz is None:
        return
    # Procesar raíz
    resultado.append(raiz.valor)
    # Procesar hijos recursivamente
    for hijo in raiz.hijos:
        recorrido_preorden(hijo, resultado)

def recorrido_postorden(raiz: TreeNode, resultado: list[int]):
    if raiz is None:
        return
    for hijo in raiz.hijos:
        recorrido_postorden(hijo, resultado)
    resultado.append(raiz.valor)

# Ejemplo
#      1
#     / \
#    2   3
#   / \
#  4   5
# raiz = TreeNode(1)
# n2 = TreeNode(2); n3 = TreeNode(3)
```



```
# raiz.agregar_hijo(n2); raiz.agregar_hijo(n3)
# n2.agregar_hijo(TreeNode(4)); n2.agregar_hijo(TreeNode(5))
# res = []
# recorrido_preorden(raiz, res)
# print(f"Pre-orden: {res}") # [1, 2, 4, 5, 3]
```

8.1.4 Centro y Diámetro de un Árbol

- **Diámetro:** El camino simple más largo entre dos nodos cualesquiera del árbol. Se puede encontrar con dos BFS:
 1. BFS desde un nodo arbitrario u para encontrar el nodo más lejano v .
 2. BFS desde v para encontrar el nodo más lejano w .
 3. La distancia entre v y w es el diámetro.
- **Centro:** El vértice (o dos vértices adyacentes) que minimiza la distancia máxima a cualquier otro nodo. Es el punto medio del diámetro.

8.2 Grafos Bipartitos

Un grafo es **bipartito** si sus vértices se pueden dividir en dos conjuntos disjuntos, U y V , de tal manera que cada arista conecte un vértice en U con uno en V . No existen aristas que conecten dos vértices dentro del mismo conjunto.

8.2.1 Caracterización

Teorema: Un grafo es bipartito si y solo si **no contiene ciclos de longitud impar**.

8.2.2 Detección de Bipartición (2-Coloreado)

Podemos verificar si un grafo es bipartito intentando colorearlo con dos colores (0 y 1) usando BFS o DFS.

1. Asignar al vértice inicial el color 0.
2. Para cada vecino:
 - Si no tiene color, asignarle el color opuesto al actual (1 - color_actual).
 - Si ya tiene color y es el mismo que el actual, **el grafo no es bipartito**.

8.2.2.1 Implementación en Python

```
from collections import deque

def es_bipartito(grafo) -> bool:
    """
    Determina si un grafo es bipartito usando 2-coloreado (BFS).
    :param grafo: Instancia de GrafoListaAdyacencia.
    """
    colores = {} # Mapa: vertice_id -> color (0 o 1)

    # Iterar sobre todos los nodos para manejar grafos no conexos
```

```

for i in range(grafo.n):
    if i in colores:
        continue

    colores[i] = 0
    cola = deque([i])

    while cola:
        u = cola.popleft()
        color_u = colores[u]

        for v, _ in grafo.obtener_vecinos(u):
            if v not in colores:
                colores[v] = 1 - color_u
                cola.append(v)
            elif colores[v] == color_u:
                # Conflicto: vecino tiene el mismo color
                return False

return True

```

8.3 Grafos Planos

Un **grafo plano** es aquel que puede ser dibujado en el plano (una hoja de papel) de tal manera que **ninguna de sus aristas se cruce**.

8.3.1 Fórmula de Euler

Para cualquier grafo plano conexo dibujado sin cruces, se cumple la relación:

$$V - E + F = 2$$

Donde: * V : Número de vértices. * E : Número de aristas. * F : Número de caras (regiones delimitadas por aristas, incluyendo la región exterior infinita).

8.3.2 Teorema de Kuratowski

¿Cómo sabemos si un grafo *no* es plano? El teorema de Kuratowski establece que un grafo es plano si y solo si no contiene un subgrafo que sea una subdivisión de:

1. K_5 : El grafo completo de 5 vértices.
2. $K_{3,3}$: El grafo bipartito completo con 3 vértices en cada conjunto.

8.3.3 Teorema de los 4 Colores

Este famoso teorema establece que cualquier grafo plano puede ser coloreado con a lo sumo **4 colores** de tal manera que no haya dos vértices adyacentes con el mismo color. Esto tiene aplicaciones directas en la cartografía (colorear mapas de países/regiones).

8.3.4 Aplicaciones

- **Diseño de Circuitos (VLSI):** Es crucial diseñar circuitos impresos donde las pistas de cobre no se crucen para evitar cortocircuitos. Se busca que el grafo del circuito sea plano o tenga el mínimo número de cruces.
- **Redes de Carreteras:** A gran escala, las redes de carreteras son casi planas (excepto por puentes y túneles). Esto permite algoritmos de navegación especializados y más rápidos que los generales.

Capítulo 9

Introducción a la Complejidad y Técnicas Avanzadas

Hasta ahora, hemos cubierto algoritmos que resuelven problemas de manera eficiente (en tiempo polinomial). Sin embargo, no todos los problemas de grafos son tan “amigables”. En este capítulo, nos adentraremos en el fascinante mundo de la complejidad computacional y exploraremos técnicas avanzadas para abordar problemas difíciles.

9.1 Clases de Complejidad: P, NP y NP-Completo

Entender qué hace que un problema sea “difícil” es crucial para no perder tiempo buscando algoritmos eficientes que probablemente no existan.

9.1.1 Definiciones Básicas

- **P (Polinomial):** La clase de problemas de decisión que pueden ser *resueltos* por un algoritmo determinista en tiempo polinomial ($O(V^k)$ para alguna constante k). Ejemplos: Camino más corto (Dijkstra), MST (Kruskal), Flujo Máximo.
- **NP (No-determinista Polinomial):** La clase de problemas cuya *solución* (si se nos da una propuesta) puede ser *verificada* en tiempo polinomial. Todo problema en P está también en NP.
- **NP-Completo:** Son los problemas “más difíciles” dentro de NP. Si pudieras resolver cualquier problema NP-Completo en tiempo polinomial, podrías resolver *todos* los problemas de NP en tiempo polinomial (demostrando que $P = NP$, uno de los mayores problemas abiertos en matemáticas).

9.1.2 Problemas NP-Completo Famosos en Grafos

1. **Problema del Viajante de Comercio (TSP):** Dado un conjunto de ciudades y distancias, encontrar la ruta más corta que visite cada ciudad exactamente una vez y regrese al inicio.

2. **Coloración de Grafos (k -Coloreado):** ¿Es posible asignar uno de k colores a cada vértice tal que ningún par de vértices adyacentes tenga el mismo color? (Para $k \geq 3$).
3. **Clique Máximo:** Encontrar el subgrafo completo más grande dentro de un grafo.
4. **Cobertura de Vértices (Vertex Cover):** Encontrar el subconjunto más pequeño de vértices tal que cada arista del grafo incida en al menos un vértice de ese subconjunto.

9.1.3 ¿Qué hacer ante un problema NP-Completo?

Si te enfrentas a uno de estos problemas en la vida real, tienes tres opciones principales: 1. **Algoritmos de Fuerza Bruta (Backtracking):** Solo viables para grafos muy pequeños. 2. **Heurísticas y Algoritmos de Aproximación:** Encontrar una solución “suficientemente buena” (cerca del óptimo) en un tiempo razonable. 3. **Casos Especiales:** Verificar si tu grafo tiene propiedades especiales (como ser un árbol o un grafo bipartito) que permitan soluciones eficientes.

9.2 2-Satisfiability (2-SAT)

El problema de satisfacibilidad booleana (SAT) es el problema NP-Completo por excelencia. Sin embargo, una variante restringida llamada **2-SAT** puede resolverse en tiempo lineal usando grafos.

9.2.1 Formulación

Dada una fórmula lógica en forma normal conjuntiva (CNF) donde cada cláusula tiene exactamente dos literales, por ejemplo:

$$(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_2 \vee \neg x_3)$$

¿Existe una asignación de valores de verdad (V/F) para las variables tal que la fórmula sea verdadera?

La clave es observar que la cláusula $(A \vee B)$ es lógicamente equivalente a las implicaciones $(\neg A \implies B)$ y $(\neg B \implies A)$.

9.2.2 Construcción del Grafo de Implicación

Construimos un grafo dirigido donde: * Los nodos son los literales (para cada variable x_i , tenemos nodos x_i y $\neg x_i$). * Las aristas representan las implicaciones. Para cada cláusula $(A \vee B)$, añadimos aristas dirigidas $(\neg A, B)$ y $(\neg B, A)$.

9.2.3 Solución con Componentes Fuertemente Conexas (SCC)

Teorema: Una instancia de 2-SAT es satisfacible si y solo si ninguna variable x y su negación $\neg x$ pertenecen a la misma Componente Fuertemente Conexas (SCC).

Si x y $\neg x$ están en la misma SCC, significa que $x \implies \dots \implies \neg x$ y $\neg x \implies \dots \implies x$, lo cual es una contradicción ($x \iff \neg x$).

Algoritmo: 1. Construir el grafo de implicación. 2. Encontrar las SCCs (usando Tarjan o Kosaraju). 3. Para cada variable x , verificar si $\text{SCC}(x) == \text{SCC}(\text{not } x)$. Si ocurre para alguna, es insatisfacible. 4. Si es satisfacible, una asignación válida se obtiene asignando Verdadero a las componentes que aparecen “más tarde” en el orden topológico inverso de las SCCs.

9.2.3.1 Implementación en Python

```
def resolver_2sat(n_vars: int, clausulas: list[tuple[int, int]]) -> bool:
    """
    Resuelve una instancia de 2-SAT.
    :param n_vars: Número de variables ( $x_1 \dots x_n$ ).
    :param clausulas: Lista de tuplas ( $u, v$ ).
                        Si  $u > 0$  representa  $x_u$ , si  $u < 0$  representa not
    ↪  $x_{|u|}$ .
    :return: True si es satisfacible, False si no.
    """
    # Mapeo de literales a nodos:
    #  $x_i \rightarrow 2*(i-1)$ , not  $x_i \rightarrow 2*(i-1) + 1$ 
    # Ejemplo:  $x_1 \rightarrow 0$ , not  $x_1 \rightarrow 1$ ,  $x_2 \rightarrow 2$ , not  $x_2 \rightarrow 3$ ...

    def literal_a_nodo(lit):
        idx = abs(lit) - 1
        return 2 * idx + (1 if lit < 0 else 0)

    def nodo_a_negacion(nodo):
        return nodo ^ 1 # 0→1, 1→0, 2→3, 3→2...

    total_nodos = 2 * n_vars
    adj = [[] for _ in range(total_nodos)]

    for u_lit, v_lit in clausulas:
        # ( $A$  or  $B$ ) equiv a ( $\text{not } A \rightarrow B$ ) y ( $\text{not } B \rightarrow A$ )
        u = literal_a_nodo(u_lit)
        v = literal_a_nodo(v_lit)
        not_u = nodo_a_negacion(u)
        not_v = nodo_a_negacion(v)

        adj[not_u].append(v)
        adj[not_v].append(u)

    # Algoritmo de Kosaraju o Tarjan para SCCs (simplificado aquí)
    # ... (implementación de SCC omitida por brevedad, ver Cap 6)
    # Suponemos una función get_sccs(adj) que retorna una lista `scc_id`
    # donde scc_id[u] es el ID del componente de u.

    # scc_id = get_sccs(adj) # Placeholder

    # Verificación:
    # for i in range(n_vars):
    #     nodo_x = 2 * i
```

```
#     nodo_not_x = 2 * i + 1
#     if scc_id[nodo_x] == scc_id[nodo_not_x]:
#         return False

return True # Si pasa todas las verificaciones
```

9.3 Heavy-Light Decomposition (HLD)

HLD es una técnica avanzada para descomponer un árbol en un conjunto de caminos (cadenas) disjuntos. Esto permite realizar consultas y actualizaciones en el camino entre dos nodos cualesquiera del árbol de manera muy eficiente: en tiempo $O(\log^2 V)$.

9.3.1 Concepto

Para cada nodo u , clasificamos sus aristas hacia los hijos en dos tipos: * **Arista Pesada (Heavy Edge)**: Conecta u con su hijo que tiene el subárbol más grande (mayor número de nodos). * **Arista Ligera (Light Edge)**: Conecta u con cualquier otro hijo.

Propiedad Clave: Cualquier camino desde la raíz a una hoja en el árbol pasará por a lo sumo $O(\log V)$ aristas ligeras. Las aristas pesadas forman cadenas continuas.

9.3.2 Aplicaciones

Al linealizar el árbol en cadenas, podemos aplicar estructuras de datos eficientes como **Segment Trees** o **Fenwick Trees** sobre cada cadena (o sobre una linealización global que respete las cadenas).

Esto nos permite resolver problemas como: * **Consulta de Máximo/Suma en Camino**: ¿Cuál es la arista de mayor peso en el camino entre el nodo u y el nodo v ? * **Actualización de Camino**: Sumar un valor k a todas las aristas en el camino entre u y v . * **Lowest Common Ancestor (LCA)**: HLD proporciona una forma de calcular el LCA.

Esta técnica es fundamental en programación competitiva y en sistemas que requieren análisis estructural rápido sobre árboles dinámicos o estáticos con consultas complejas.

9.4 Grafos Aleatorios y Redes de Pequeño Mundo

Finalmente, una breve mención a modelos de grafos que describen redes reales:

- **Grafos Aleatorios (Erdős-Rényi)**: Cada par de nodos se conecta con una probabilidad p . Útil como modelo base, pero no captura la estructura de redes sociales reales.

- **Redes de Pequeño Mundo (Watts-Strogatz):** Modelan el fenómeno de “seis grados de separación”. Tienen alto coeficiente de agrupamiento (mis amigos son amigos entre sí) y caminos cortos promedios.
- **Redes Libres de Escala (Barabási-Albert):** Algunos nodos (“hubs”) tienen muchísimas conexiones, siguiendo una ley de potencia. Modelan Internet, citas académicas y redes sociales mejor que los grafos aleatorios puros.

Capítulo 10

Aplicaciones Prácticas y Estudios de Caso

Hemos recorrido un largo camino desde las definiciones básicas de vértices y aristas hasta algoritmos complejos de flujo máximo y conectividad. En este capítulo final, aterrizaremos todos estos conceptos explorando cómo la teoría de grafos es la fuerza invisible que impulsa muchas de las tecnologías y sistemas que utilizamos a diario.

10.1 Análisis de Redes Sociales (Social Network Analysis - SNA)

Las redes sociales son quizás el ejemplo más intuitivo de un grafo gigante. El análisis de estas estructuras permite entender dinámicas sociales, propagación de información y la importancia de los individuos en un grupo.

10.1.1 Métricas de Centralidad

¿Quién es la persona más “importante” en una red? La respuesta depende de cómo definamos “importante”.

- **Centralidad de Grado (Degree Centrality):**
 - **Definición:** El número de conexiones directas que tiene un nodo.
 - **Significado:** Popularidad inmediata. Quien tiene más amigos o seguidores.
 - **Cálculo:** Simplemente $\text{grado}(v)$.
- **Centralidad de Intermediación (Betweenness Centrality):**
 - **Definición:** Cuantifica la frecuencia con la que un nodo actúa como un puente a lo largo del camino más corto entre otros dos nodos.
 - **Significado:** Control del flujo de información. Alguien que conecta grupos dispares (ej. el único amigo en común entre tus amigos del colegio y tus amigos del trabajo).
 - **Algoritmo:** Requiere calcular caminos más cortos entre todos los pares (BFS/Dijkstra/Floyd-Warshall).
- **PageRank (Centralidad de Vector Propio):**

- **Definición:** La importancia de un nodo depende de la importancia de sus vecinos.
- **Origen:** El algoritmo original de Google para clasificar páginas web. Una página es importante si muchas páginas importantes la enlazan.
- **Cálculo:** Iterativo o mediante álgebra lineal (vectores propios).

10.1.2 Detección de Comunidades

Identificar grupos densamente conectados dentro de la red. * **Algoritmo de Louvain:** Optimiza la “modularidad” para encontrar comunidades jerárquicas. * **Algoritmo de Girvan-Newman:** Elimina iterativamente las aristas con mayor “betweenness” para separar el grafo en comunidades naturales.

10.2 Mapas y Navegación

Aplicaciones como Google Maps, Waze o Uber dependen críticamente de algoritmos de grafos eficientes.

10.2.1 Modelado del Problema

- **Nodos:** Intersecciones de calles.
- **Aristas:** Segmentos de calle que conectan intersecciones.
- **Pesos:**
 - **Distancia:** Longitud física del segmento.
 - **Tiempo:** Función de la distancia, límite de velocidad y **tráfico en tiempo real**.
 - **Costo:** Peajes, consumo de combustible.

10.2.2 Algoritmos en Producción

- **A* (A-Star):** Es el estándar para encontrar rutas punto a punto. Utiliza la distancia geográfica (euclidiana o Haversine) como heurística para guiar la búsqueda hacia el destino, explorando órdenes de magnitud menos nodos que Dijkstra puro.
 - **Jerarquías de Contracción (Contraction Hierarchies - CH):**
 - Técnica de preprocesamiento avanzada.
 - Añade “atajos” al grafo que representan caminos rápidos precalculados.
 - Permite consultas de ruta en microsegundos en grafos de escala continental, mucho más rápido que A* estándar.
-

10.3 Compiladores y Sistemas Operativos

El software que construye y ejecuta otro software está lleno de grafos.

10.3.1 Resolución de Dependencias

Gestores de paquetes (pip, npm, apt) y sistemas de construcción (Make, Gradle, Bazel) modelan los artefactos y sus dependencias como un **Grafo Dirigido Acíclico (DAG)**.

- **Orden de Instalación/Compilación:** Se obtiene mediante un **Ordenamiento Topológico**. Si A depende de B, B debe procesarse antes que A.
- **Paralelismo:** Los nodos que no tienen dependencias entre sí (o cuyas dependencias ya se cumplieron) pueden procesarse en paralelo.
- **Detección de Ciclos:** Si se detecta un ciclo (A depende de B, B depende de A), el sistema reporta un error de “dependencia circular”.

10.3.2 Asignación de Registros (Register Allocation)

Cuando un compilador traduce código fuente a código máquina, debe asignar un número ilimitado de variables del programa a un número limitado de registros físicos de la CPU.

- **Grafo de Interferencia:**
 - **Nodos:** Variables del programa.
 - **Aristas:** Conectan dos variables si están “vivas” simultáneamente (sus tiempos de vida se solapan).
 - **Coloración de Grafos:** El problema se modela como encontrar una k -coloración del grafo, donde k es el número de registros disponibles. Si dos nodos están conectados (interfieren), deben tener colores (registros) distintos. Si no se puede colorear con k colores, algunas variables deben moverse a la memoria RAM (“spilling”).
-

10.4 Biología Computacional

La bioinformática utiliza grafos para resolver puzzles biológicos complejos.

10.4.1 Ensamblaje de Genomas

Los secuenciadores de ADN modernos leen fragmentos cortos de ADN (“reads”). El problema es reconstruir la secuencia original completa a partir de millones de estos fragmentos solapados.

- **Grafo de De Bruijn:**
 - Los nodos representan subsecuencias de longitud $k - 1$ (k -mers).
 - Las aristas representan solapamientos.
 - El ensamblaje del genoma se modela como encontrar un **Camino Euleriano** (un camino que visita cada arista exactamente una vez) en este grafo.

10.4.2 Redes de Interacción de Proteínas (PPI)

Grafos donde los nodos son proteínas y las aristas representan interacciones físicas o funcionales. Analizar estas redes ayuda a identificar funciones de

proteínas desconocidas (por asociación con vecinos conocidos) y a descubrir nuevas dianas para fármacos.

10.5 Conclusión

La teoría de grafos es mucho más que una rama de las matemáticas discretas; es un lenguaje universal para describir relaciones y estructuras.

Desde encontrar la ruta más rápida a casa hasta descifrar el genoma humano, pasando por compilar tu código y recomendarte tu próxima serie favorita, los grafos están en el corazón de la resolución de problemas modernos.

Esperamos que este libro te haya proporcionado las herramientas teóricas y prácticas para ver el mundo a través de los grafos y aplicar estos poderosos algoritmos en tus propios proyectos.

¡Feliz codificación!